

《神经网络与深度学习》

循环神经网络

周晓飞

xiaofei-zhou.com

自动化学院

参考资料

▶ 网络资料

▶ An Introduction to Recurrent Neural Networks

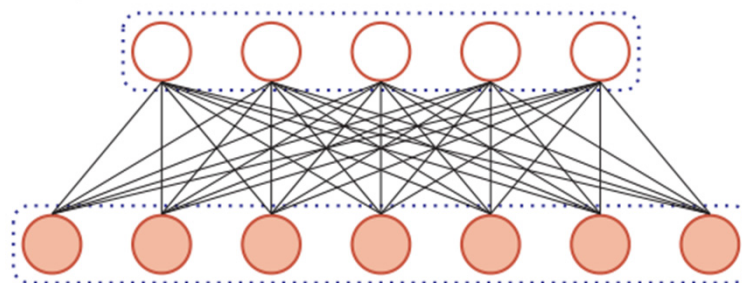
▶ <https://medium.com/explore-artificial-intelligence/an-introduction-to-recurrent-neural-networks-72c97bf0912>

▶ Recurrent Neural Networks

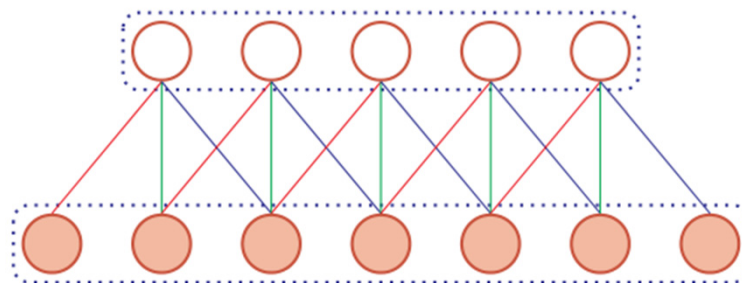
▶ <https://towardsdatascience.com/recurrent-neural-networks-d4642c9bc7ce>

前馈网络

- ▶ 连接存在层与层之间，每层的节点之间是无连接的。（无循环）
- ▶ 输入和输出的维数都是固定的，不能任意改变。无法处理变长的序列数据。



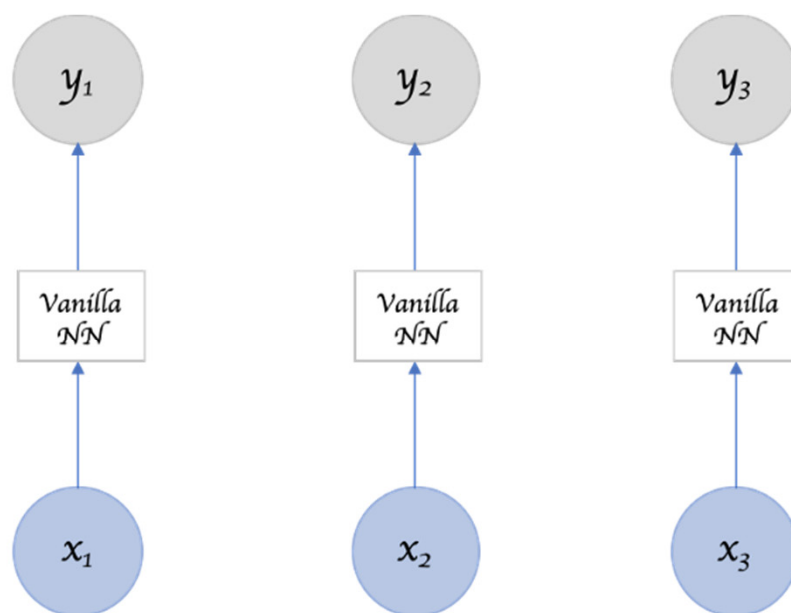
(a) 全连接层



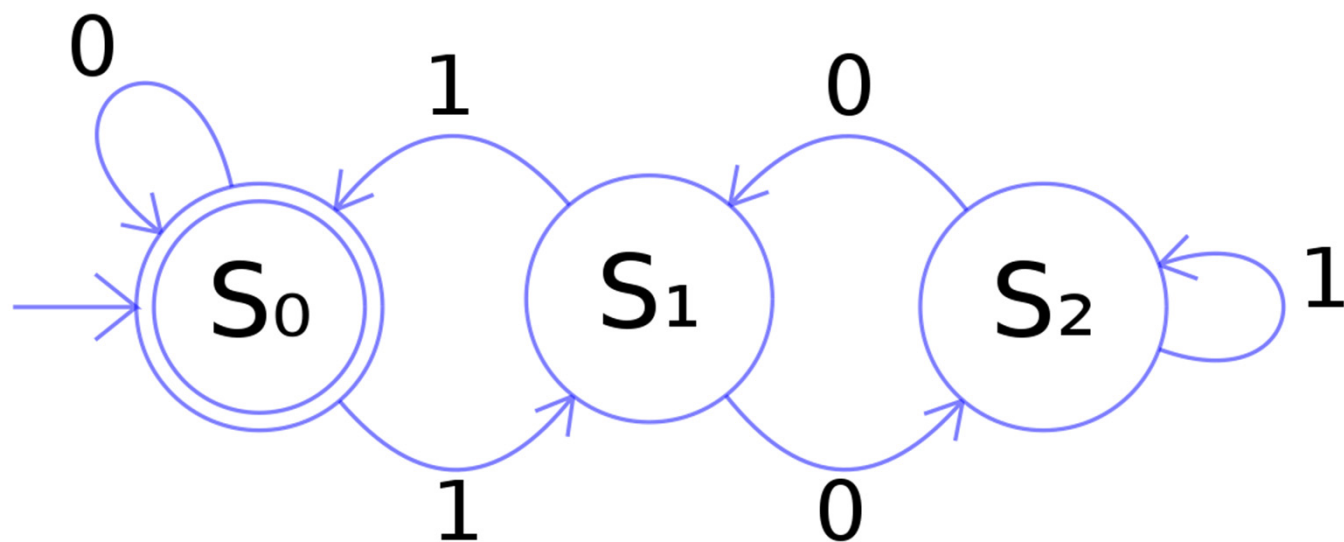
(b) 卷积层

前馈网络

- 假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。



有限状态自动机 (Finite Automata)



如何用**FNN**去模拟一个有限状态自动机?

循环神经网络 Recurrent Neural Network RNN

- ▶ 具有短期记忆能力的神经网络。
 - ▶ 不但可以接受其他神经元的信息，也可以接受自身信息，形成具有环路的网络结构。
 - ▶ RNN已经广泛应用于语音识别、语言模型及自然语言生成等任务上。
- ▶ RNN的参数可以通过随时间反向传播算法 [werbos,1990] 来学习。
 - ▶ 存在长程依赖问题，可引入门控机制进行缓解（Gate Mechanism）
 - ▶ 递归神经网络和图网络

1 给网络增加记忆能力

1 延时神经网络

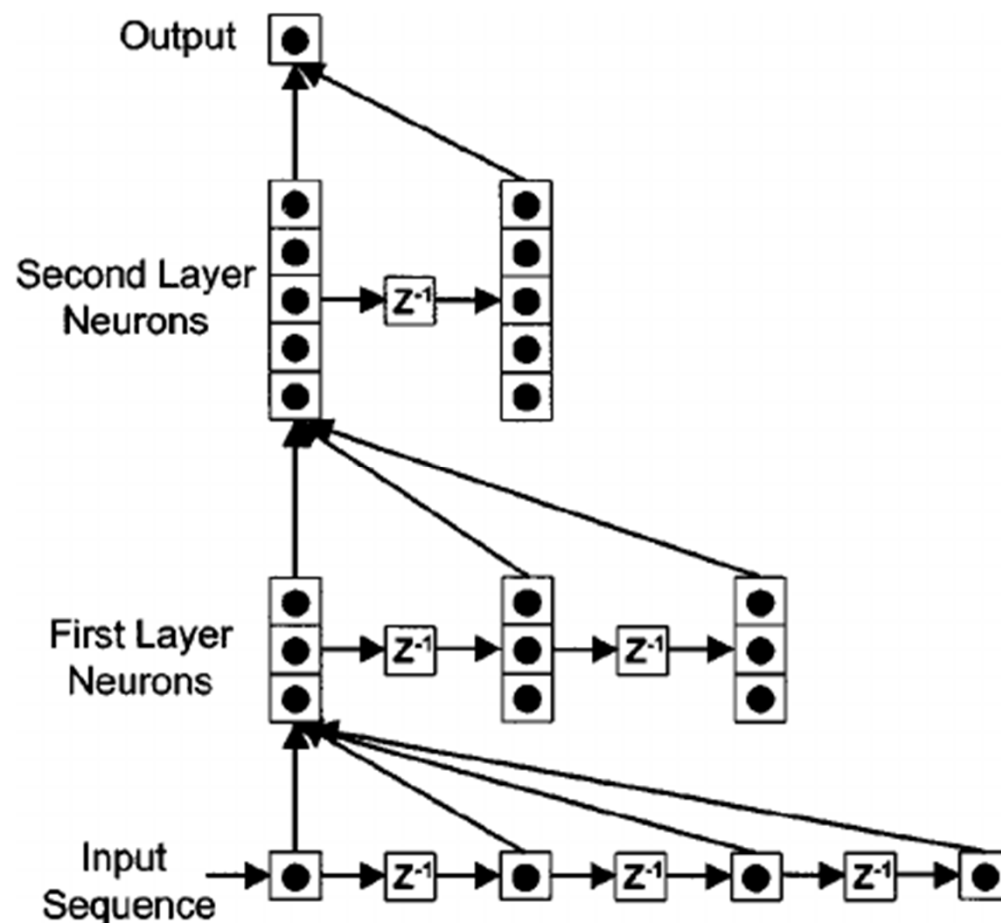
► 延时神经网络 (Time Delay Neural Network, TDNN)

- 建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）
- 在前馈神经网络中的非输出层都增加一个延时器，记录神经元的最近几次活性值。在第t时刻，第l层神经元的活性值依赖于第l-1层神经元的最近P个时刻的活性值，即

$$\mathbf{h}_t^{(l+1)} = f(\mathbf{h}_t^{(l)}, \mathbf{h}_{t-1}^{(l)}, \dots, \mathbf{h}_{t-p}^{(l)})$$

- 通过延时器，前馈网络具有了短期记忆能力

延时神经网络



https://www.researchgate.net/publication/12314435_Neural_system_identification_model_of_human_sound_localization

2 有外部输入的非线性自回归模型

▶ 自回归模型 (Autoregressive Model, AR)

- ▶ 一类时间序列模型，用一个变量 y_t 的历史信息来预测自己

$$y_t = w_0 + \sum_{i=1}^p w_i y_{t-i} + \epsilon_t$$

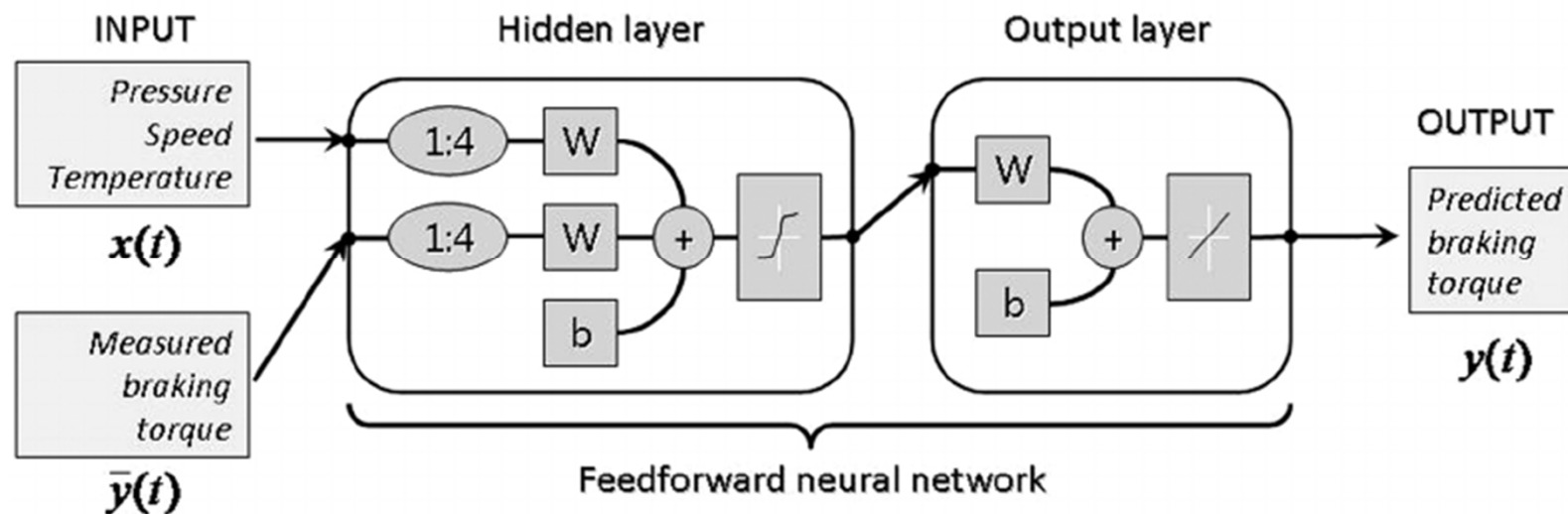
- ▶ $\epsilon_t \sim N(0, \sigma^2)$ 为第 t 个时刻的噪声

▶ 有外部输入的非线性自回归模型 (Nonlinear Autoregressive with Exogenous Inputs Model, NARX)

- ▶ 每个时刻 t ，都有一个外部输入 x_t ，产生一个输出 y_t ；NARX通过一个延时器记录最近 p 次的外部输入和最近 q 次的输出

$$y_t = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-p}, \mathbf{y}_{t-1}, \mathbf{y}_{t-2}, \dots, \mathbf{y}_{t-q})$$

2 有外部输入的非线性自回归模型



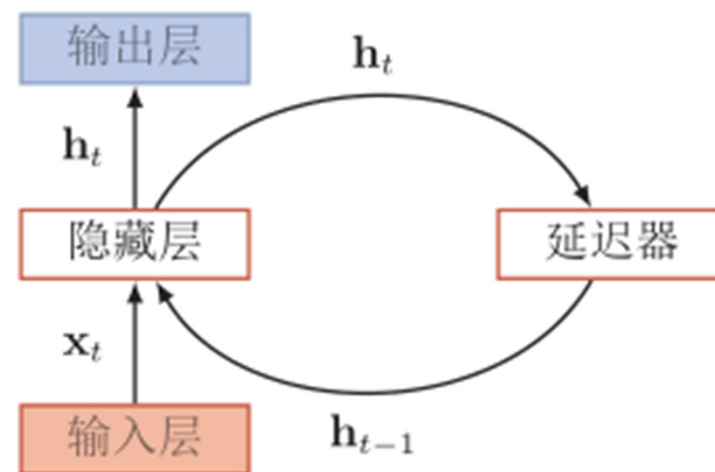
https://www.researchgate.net/publication/234052442_Braking_torque_control_using_reccurent_neural_networks

3 循环神经网络

- ▶ 循环神经网络 Recurrent Neural Network, RNN
 - ▶ 通过使用带自反馈的神经元，能够处理任意长度的时序数据
 - ▶ 给定一个输入序列 $\mathbf{x}_{1:T} = (x_1, x_2, \dots, x_t, \dots, x_T)$ ，循环神经网络通过下面的公式更新带反馈边的隐藏层的活性值 h_t

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

- ▶ 隐藏层活性值 h_t 亦被称作状态 (state) 或隐状态 (Hidden state)



循环神经网络

► 循环神经网络

- 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的序列。
- 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- 循环神经网络已经被广泛应用于语音识别、语言模型以及自然语言生成等任务上。

2 简单循环网络

简单循环神经网络

- ▶ 简单循环神经网络 Simple Recurrent Network, SRN
 - ▶ 只有一个隐藏层的神经网络；增加了从隐藏层到隐藏层的反馈连接
 - ▶ 令 $\mathbf{x}_t \in \mathbb{R}^M$ 表示在时刻 t 时网络的输入, $\mathbf{h}_t \in \mathbb{R}^D$ 表示隐藏层的状态（即隐藏层神经元活性值），则 $\mathbf{h}_t$ 不仅和当前时刻的输入 $\mathbf{x}_t$ 相关，也和上一时刻的隐状态 $\mathbf{h}_{t-1}$ 相关。简单循环网络在时刻 t 的更新公式为

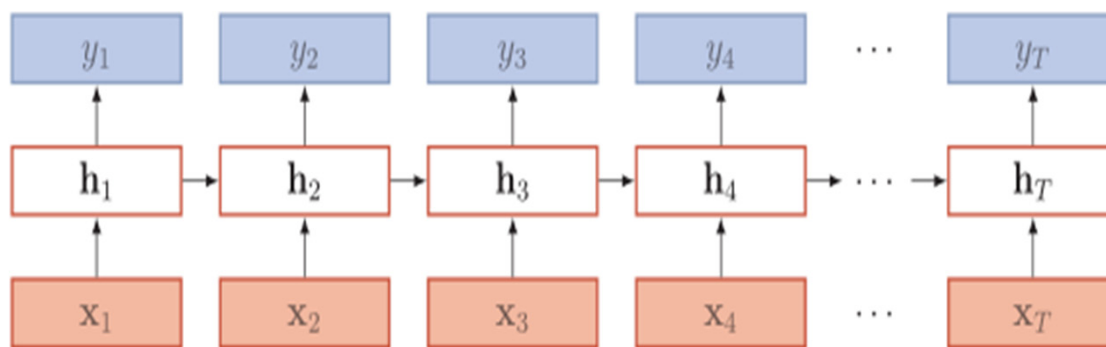
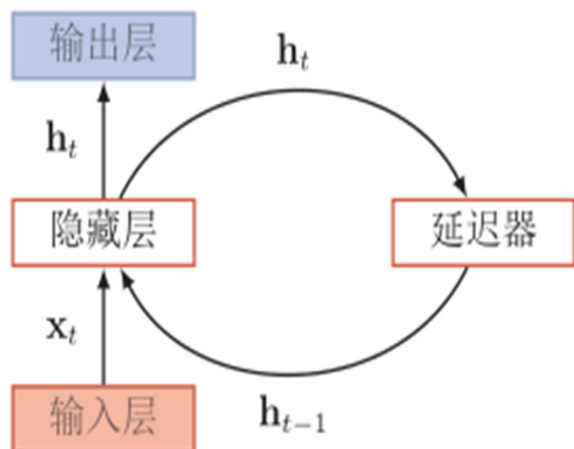
$$\mathbf{z}_t = \mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b}$$

$$\mathbf{h}_t = f(\mathbf{z}_t)$$

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

按时间展开

- 如果我们把每个时刻的状态都看作前馈神经网络的一层，循环神经网络可以看作在时间维度上权值共享的神经网络。

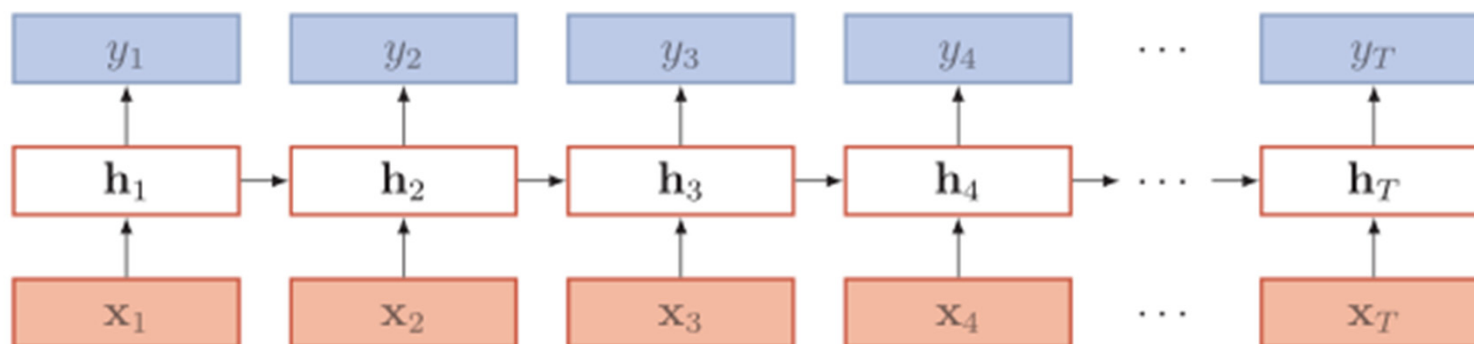


1 循环神经网络的计算能力

- 定义一个全连接的循环神经网络，其输入为 $\mathbf{x}_t$ ，输出为 y_t

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

$$y_t = \mathbf{V}\mathbf{h}_t$$



通用近似定理

- ▶ 一个完全连接的循环网络是任何非线性动力系统的近似器。

定理 6.1 – 通用近似定理 [Haykin, 2009]: 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元，它可以以任意的准确率去近似任何一个非线性动力系统

$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{x}_t), \quad (6.10)$$

$$\mathbf{y}_t = o(\mathbf{s}_t), \quad (6.11)$$

其中 $\mathbf{s}_t$ 为每个时刻的隐状态， $\mathbf{x}_t$ 是外部输入， $g(\cdot)$ 是可测的状态转换函数， $o(\cdot)$ 是连续输出函数，并且对状态空间的紧致性没有限制。

2 图灵完备

- ▶ 图灵完备 (Turing Completeness) 是指一种数据操作规则，比如一种计算机编程语言，可以实现图灵机的所有功能，解决所有的可计算问题。

定理 6.2 – 图灵完备 [Siegelmann and Sontag, 1991]: 所有的图灵机都可以被一个由使用 sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

- ▶ 一个完全连接的循环神经网络可以近似解决所有的可计算问题。

3 应用到机器学习

应用到机器学习

- ▶ 序列到类别
- ▶ 同步的序列到序列模式
- ▶ 异步的序列到序列模式

1 序列到类别模式

► 序列到类别模式主要用于序列数据的分类问题：输入为序列，输出为类别

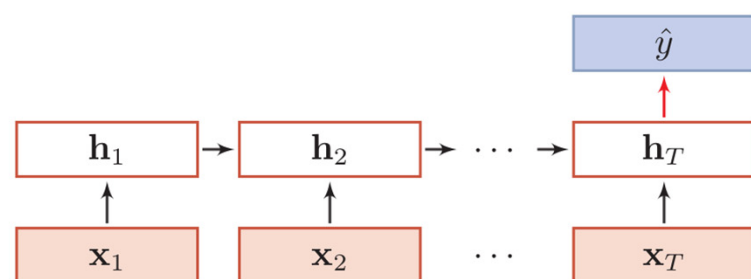
- 假设样本 $\mathbf{x}_{1:T} = (x_1, x_2, \dots, x_t, \dots, x_T)$ 为一个长度为T的序列，输出为一个类别 $y \in \{1, \dots, C\}$ 。将 $\mathbf{x}$ 按照不同时刻输入到循环神经网络中，得到不同时刻的隐藏状态 $h_1, \dots, h_T$ 。我们可以将 h_T 看作整个序列的最终表示，并输入给分类器 $g()$ 进行分类，

即

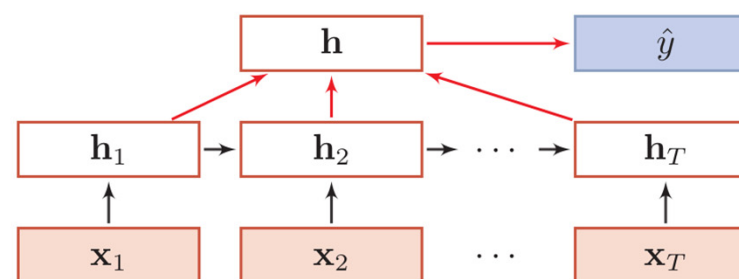
$$\hat{y} = g(h_T)$$

- 如图(a)所示， $g()$ 可以是简单线性分类器或复杂分类器
- 亦可将整个序列所有状态进行平均，以此作为整个序列的状态表示，如图 (b) 所示

$$\hat{y} = g\left(\frac{1}{T} \sum_{t=1}^T h_t\right)$$



(a) 正常模式



(b) 按时间进行平均采样模式

1 序列到类别模式

► 情感分类

带着愉悦的心情
看了这部电影

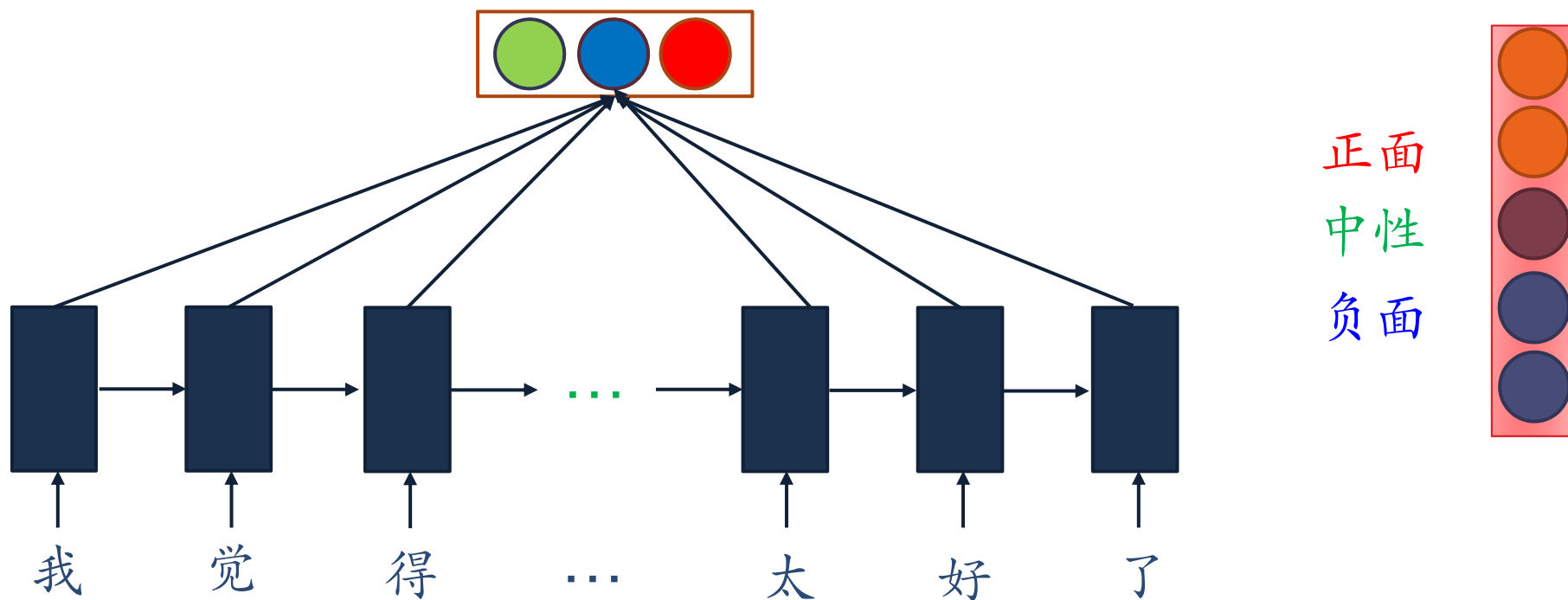
Positive (正面)

这部电影太糟了

Negative (负面)

这部电影很棒

Positive (正面)

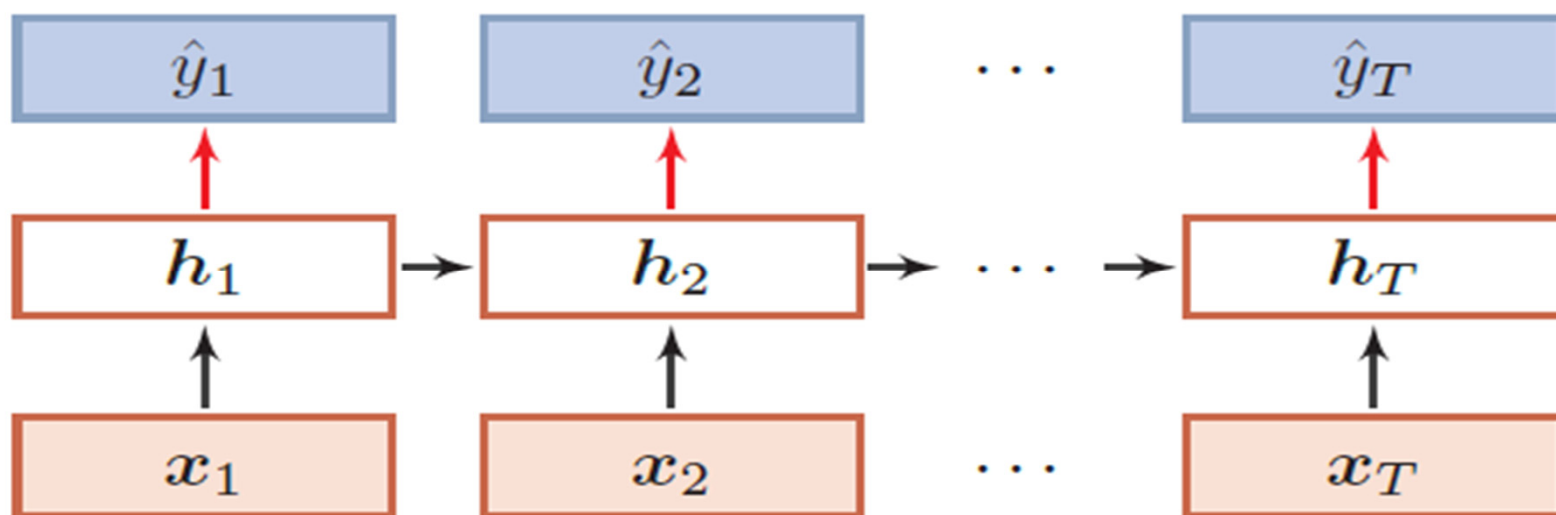


2 同步的序列到序列模式

► 同步的序列到序列模式主要用于序列序列标注 (Sequence Labeling) 任务

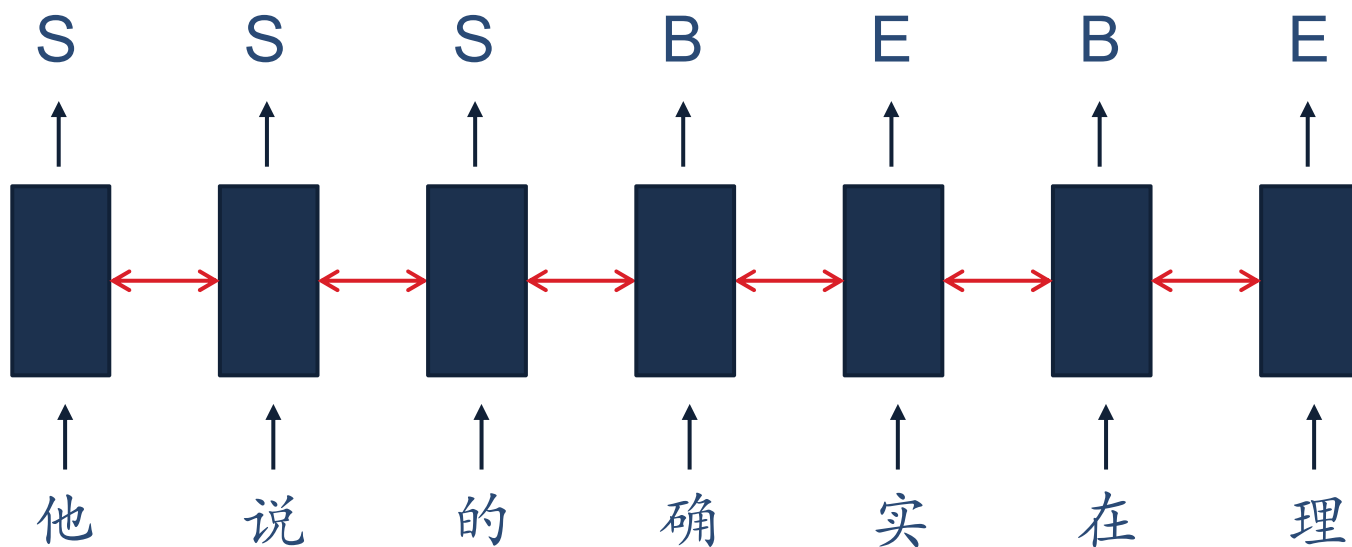
- 即每一时刻都有输入和输出，输入序列和输出序列的长度。词性标注中，每个单词都标注其对应的词性标签。
- 输入长度为T的序列 $\mathbf{x}_{1:T} = (x_1, \dots, x_T)$ ，输出为 $\mathbf{y}_{1:T} = (y_1, \dots, y_T)$ ，样本按不同时刻输入RNN中，得到对应的隐状态 $h_1, \dots, h_T$ 。每个隐状态 h_t 代表了当前时刻和历史时刻的信息，并输入给分类器 $g(\cdot)$ 得到当前时刻的标签 $\hat{y}_t$ ，即

$$\hat{y}_t = g(h_t), \quad \forall t \in [1, T]$$



2 同步的序列到序列模式

► 中文分词

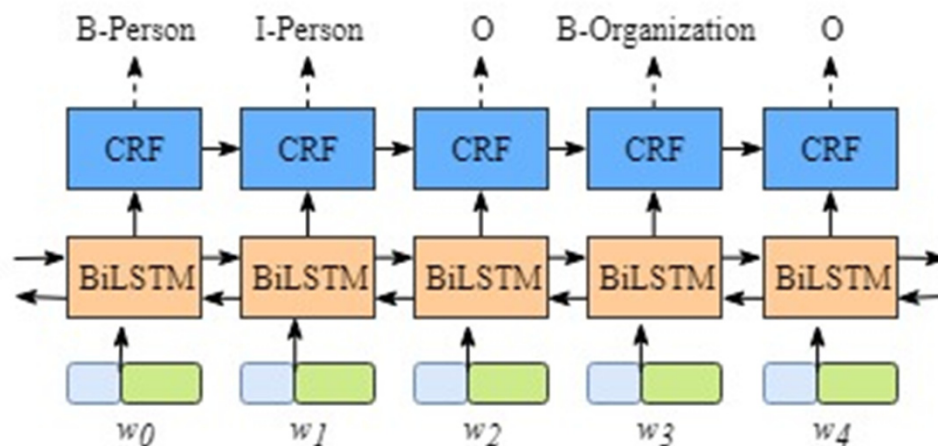


2 同步的序列到序列模式

► 信息抽取(Information Extraction, IE)

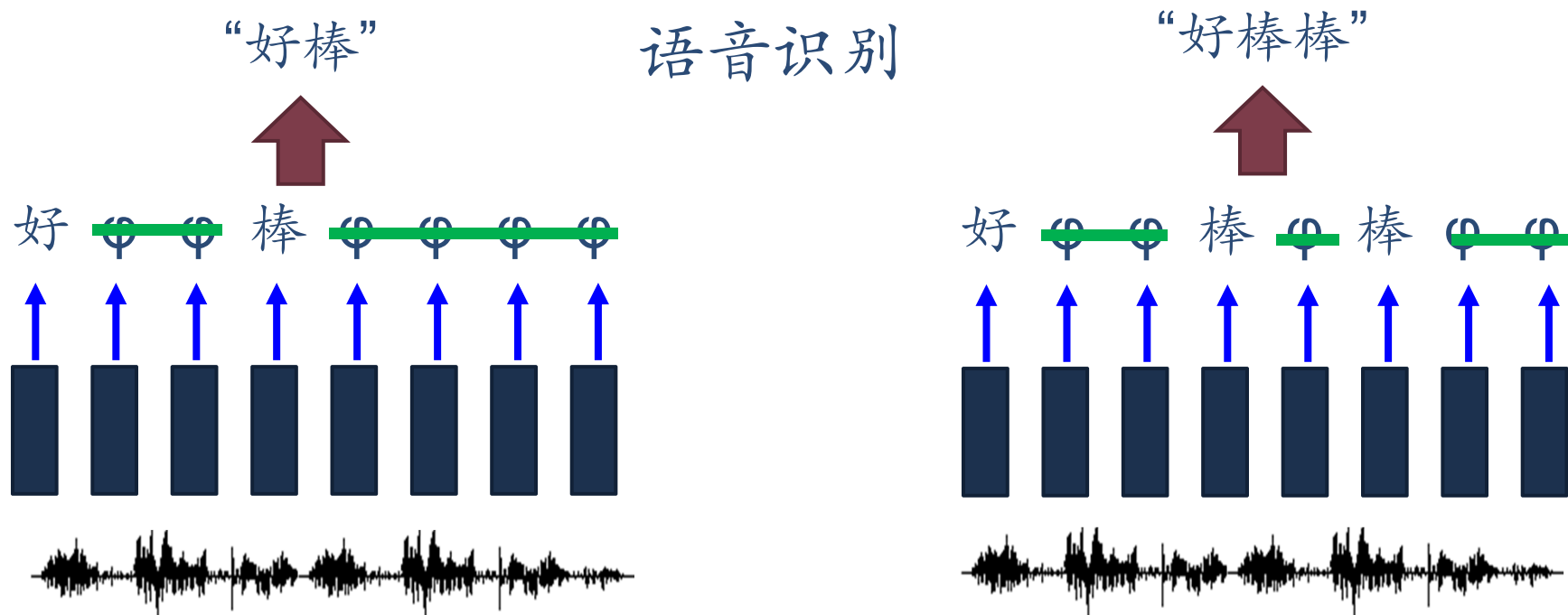
- 从无结构的文本中抽取结构化的信息，形成知识

小米创始人雷军表示，该公司2015年营收达到780亿元人民币，较2014年的743亿元人民币增长了5%。



2 同步的序列到序列模式

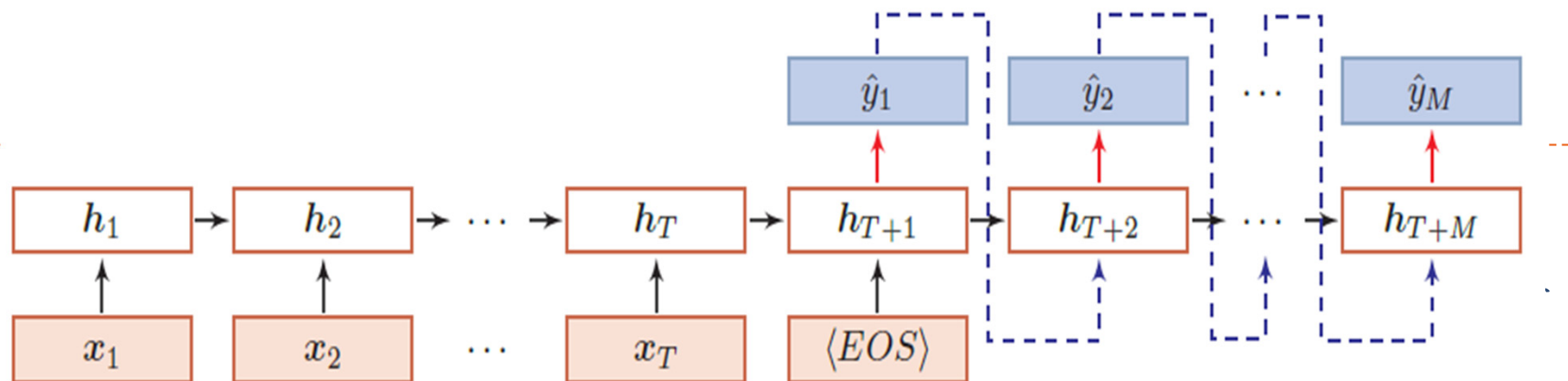
- ▶ Connectionist Temporal Classification (CTC) [Alex Graves, ICML' 06][Alex Graves, ICML' 14][Haşim Sak, Interspeech' 15][Jie Li, Interspeech' 15][Andrew Senior, ASRU' 15]



3 异步的序列到序列模式

▶ 异步的序列到序列模式也称为编码器-解码器 (Encoder-Decoder) 模型

- ▶ 输入序列和输出序列无需严格的对应关系，也不需要保持相同的长度。比如在机器翻译中，输入为源语言的单词序列，输出为目标语言的单词序列。
- ▶ 输入长度为 T 的序列 $\mathbf{x}_{1:T} = (x_1, \dots, x_T)$ ，输出为 $\mathbf{y}_{1:M} = (y_1, \dots, y_M)$ ，异步模式一般通过先**编码**后**解码**的方式实现
- ▶ 样本 $\mathbf{x}$ 按不同时刻输入编码器RNN中，得到其编码 $\hat{\mathbf{h}}_T$ 。然后在使用解码器RNN，得到输出序列 $\mathbf{y}_{1:M}$ 。
- ▶ 为建立输出序列之间的依赖关系，解码器常采用非线性自回归模型。



$$h_t = f_1(h_{t-1}, x_t), \forall t \in [1, T]$$

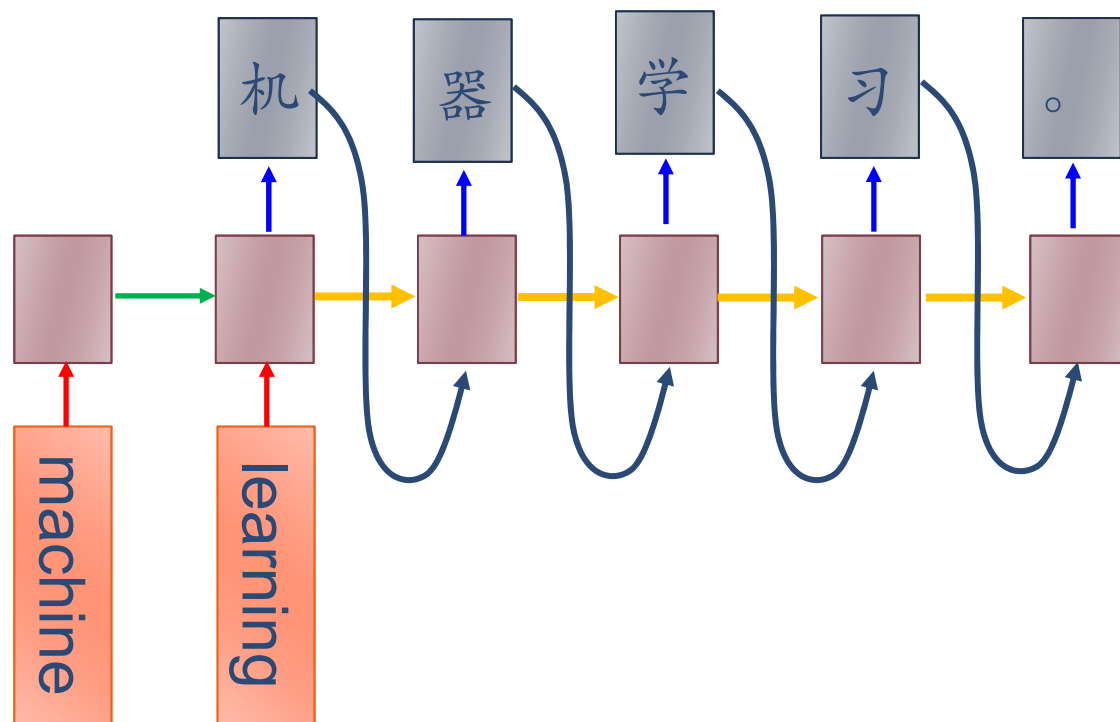
$$h_{T+t} = f_2\left(h_{T+t-1}, \hat{y}_{t-1}\right), \forall t \in [1, M]$$

$$\hat{y}_t = g(h_{T+t}), \forall t \in [1, M]$$

► 解码器通常采用自回归模型

异步的序列到序列模式

► 机器翻译



4 参数学习

参数学习

► 机器学习

► 给定一个训练样本 $(\mathbf{x}, \mathbf{y})$ ，其中

► $\mathbf{x} = (x_1, \dots, x_T)$ 为长度是 T 的输入序列，

► $\mathbf{y} = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

► 时刻 t 的瞬时损失函数为

$$\mathcal{L}_t = \mathcal{L}(\mathbf{y}_t, g(\mathbf{h}_t)),$$

► 整个序列的总损失函数

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

参数学习

- ▶ 整个序列的损失函数关于参数U的梯度

$$\frac{\partial L}{\partial \mathbf{U}} = \sum_{t=1}^T \frac{\partial L_t}{\partial \mathbf{U}}$$

- ▶ 即 每个时刻损失 L_t 对参数U的偏导数之和
- ▶ 循环神经网络有两种计算梯度的方式：随时间反向传播（BPTT）和实时循环学习（RTRL）

1 随时间反向传播BPTT

- ▶ BPTT算法将RNN看做一个展开的多层前馈网络，其中“每一层”对应循环网络中的“每个时刻”。这样RNN可以按照前馈网络中的反向传播算法计算参数梯度。
- ▶ 在“展开”的前馈网络中，所有层的参数是共享的，因此，参数的值是梯度是所有“展开层”得参数梯度之和
- ▶ 计算偏导数 $\frac{\partial L}{\partial U}$
 - ▶ 参数U和隐藏层在每个时刻k ($1 \leq k < t$) 的净输入 z_k 有关，因此，第t时刻的损失函数 L_t 关于参数 U_{ij} 的梯度为

$$z_k = U h_{k-1} + W x_k + b$$

$$\frac{\partial L}{\partial \mathbf{u}_{ij}} = \sum_{k=1}^t \frac{\partial^+ z_k}{\partial \mathbf{u}_{ij}} \frac{\partial L_t}{\partial z_k}$$

$$\text{其中, } \frac{\partial^+ z_k}{\partial \mathbf{u}_{ij}} = \left[0, \dots, [\mathbf{h}_{k-1}]_j, \dots, 0 \right] \stackrel{\Delta}{=} \mathbf{I}_i \left([\mathbf{h}_{k-1}]_j \right)$$

1 随时间反向传播BPTT

- 定义误差项为第t时刻的损失对第k时刻隐藏神经层的净输入 z_k 的导数，则当 $1 \leq k < t$ 时

$$\begin{aligned}\delta_{t,k} &= \frac{\partial L_t}{\partial z_k} \\ &= \frac{\partial h_k}{\partial z_k} \frac{\partial z_{k+1}}{\partial h_k} \frac{\partial L_t}{\partial z_{k+1}} \\ &= \text{diag}(f'(z_k)) U^T \delta_{t,k+1}\end{aligned}$$

- 将上述两个公式合并，得到

$$\frac{\partial L_t}{\partial u_{ij}} = \sum_{k=1}^t [\delta_{t,k}]_i [\mathbf{h}_{k-1}]_j$$

$$\frac{\partial L_t}{\partial \mathbf{U}} = \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

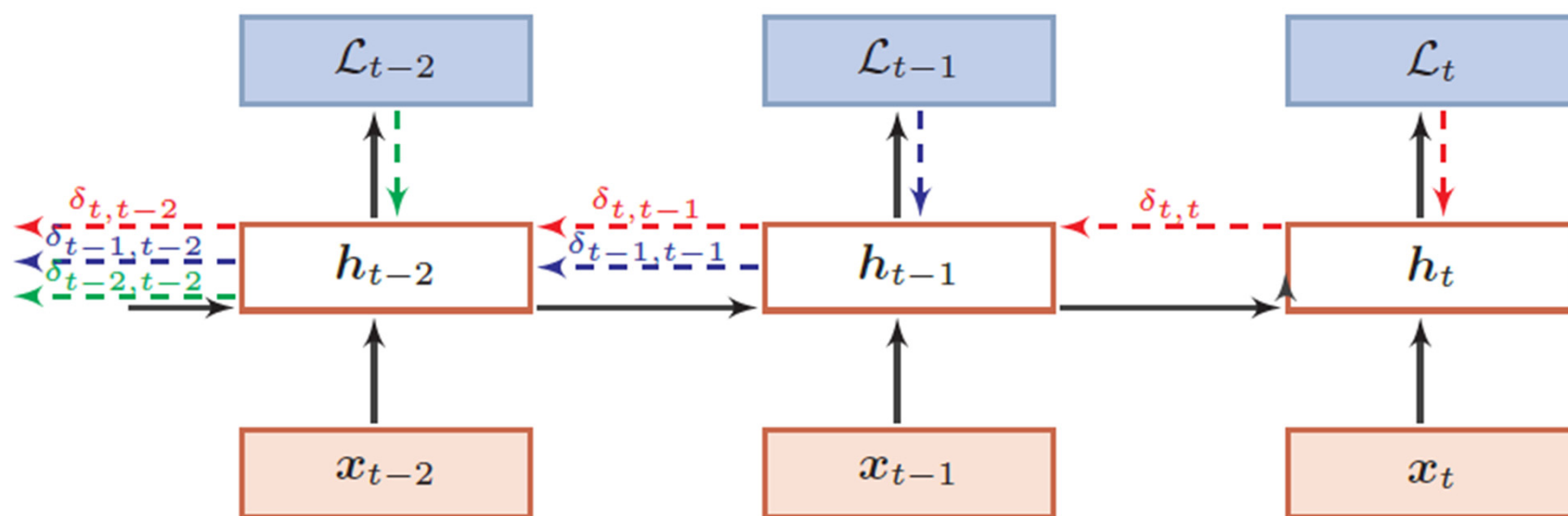


图 6.6 误差项随时间反向传播算法示例

1 随时间反向传播BPTT

▶ 参数梯度

- ▶ 计算整个序列的损失函数L关于参数U的梯度

$$\frac{\partial L}{\partial \mathbf{U}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

- ▶ L关于权重W和偏置b的梯度为

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{x}_k^T \quad \frac{\partial L}{\partial \mathbf{b}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k}$$

▶ 计算复杂度

- ▶ BPTT算法中，参数梯度计算需要完整的“前向”和“后向”计算后才能得到并进行参数更新

5 长程依赖问题

梯度消失/爆炸

► 梯度

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

► 其中

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \underbrace{\left(\text{diag}(f'(\mathbf{z}_{\tau})) U^T \right)}_r \delta_{t,t}$$

由于梯度爆炸 ($r > 1$) 或梯度消失 ($r < 1$) 问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题 (Long-Term Dependencies)，即如果时刻 t 的输出 y_t 依赖于 k 时刻的输入 x_k ，当时间间隔 $t-k$ 比较大时，简单神经网络很难建立这种长距离的依赖关系。

长程依赖问题

▶ 循环神经网络在时间维度上非常深!

- ▶ 梯度消失或梯度爆炸

▶ 如何改进?

- ▶ 梯度爆炸问题

 - ▶ 权重衰减

 - ▶ 梯度截断

- ▶ 梯度消失问题

 - ▶ 改进模型

长程依赖问题

► 梯度爆炸

- 权重衰减，即通过给参数增加l1或者l2范数的正则项来限制参数的取值范围，从而使得 $r \leq 1$.
- 梯度截断，一种启发式方法，当梯度的模大于一定阈值时，就将它截断为一个较小的数

► 梯度消失

- 改变模型，循环边改为线性依赖关系，比如令 $U=I$, $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = I$ ，即

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta)$$

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

其中， $g()$ 是一个非线性函数

长程依赖问题

► 梯度消失

► 增加非线性

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$

这样 $\mathbf{h}_t$ 和 $\mathbf{h}_{t-1}$ 之间既有线性关系，也有非线性关系

► 存在的问题

► 梯度爆炸，即依据上式计算误差项时 $\delta_{t,k} = \text{diag}(f'(z_k)) \mathbf{U}^T \delta_{t,k+1}$ 出现梯度过大

► 记忆容量，即随着 $\mathbf{h}_t$ 的不断累积存储新的输入信息，会发生饱和现象（ $g()$ 为饱和型函数）

6 基于门控的循环神经网络 Gated RNN

1 长短期记忆网络 LSTM

- ▶ 长期记忆网络 (Long Short-Term Memory Network, LSTM) 在公式 $\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$ 的基础上, LSTM 主要改进以下两个方面:

- ▶ 1) 新的内部状态, 即LSTM引入一个新的内部状态 $\mathbf{c}_t \in \mathbb{R}^D$ 专门进行线性的循环信息传递, 同时非线性的输出信息给隐藏层的外部状态 $\mathbf{h}_t \in \mathbb{R}^D$ 。内部状态 $\mathbf{c}_t$ 通过下面的公式计算:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- ▶ 其中 $\mathbf{f}_t \in [0, 1]^D$, $\mathbf{i}_t \in [0, 1]^D$ 和 $\mathbf{o}_t \in [0, 1]^D$ 为三个门gate来控制信息传递的路径, $\odot$ 为向量元素相乘, $\mathbf{c}_{t-1}$ 为上一时刻的记忆单元

1 长短期记忆网络 LSTM

- ▶ $\tilde{\mathbf{c}}_t \in \mathbb{R}^D$ 是通过非线性函数得到的候选状态，即

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c)$$

- ▶ 在每个时刻 t , LSTM 网络的内部状态 $\mathbf{c}_t$ 记录了到当前时刻为止的历史信息。

- ▶ 2) 门控机制，前述两个公式（如下）分别有输入门 $\mathbf{i}_t$ 、遗忘门 $\mathbf{f}_t$ 和输出门 $\mathbf{o}_t$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- ▶ 此三门的作用：

- ✓ 遗忘门 $\mathbf{f}_t$ 控制上一时刻的内部状态 $\mathbf{c}_{t-1}$ 需要遗忘多少信息
- ✓ 输入门 $\mathbf{i}_t$ 控制当前时刻的候选状态 $\tilde{\mathbf{c}}_t$
- ✓ 输出门 $\mathbf{o}_t$ 控制当前时刻的内部状态 $\mathbf{c}_t$ 有多少信息需要输出给外部状态 $\mathbf{h}_t$

1 长短期记忆网络 LSTM

- ▶ LSTM网络中的“门”是一种“软”门，取值为 $(0,1)$ 之间，即以一定的比例允许信息通过。三个门的计算方式：

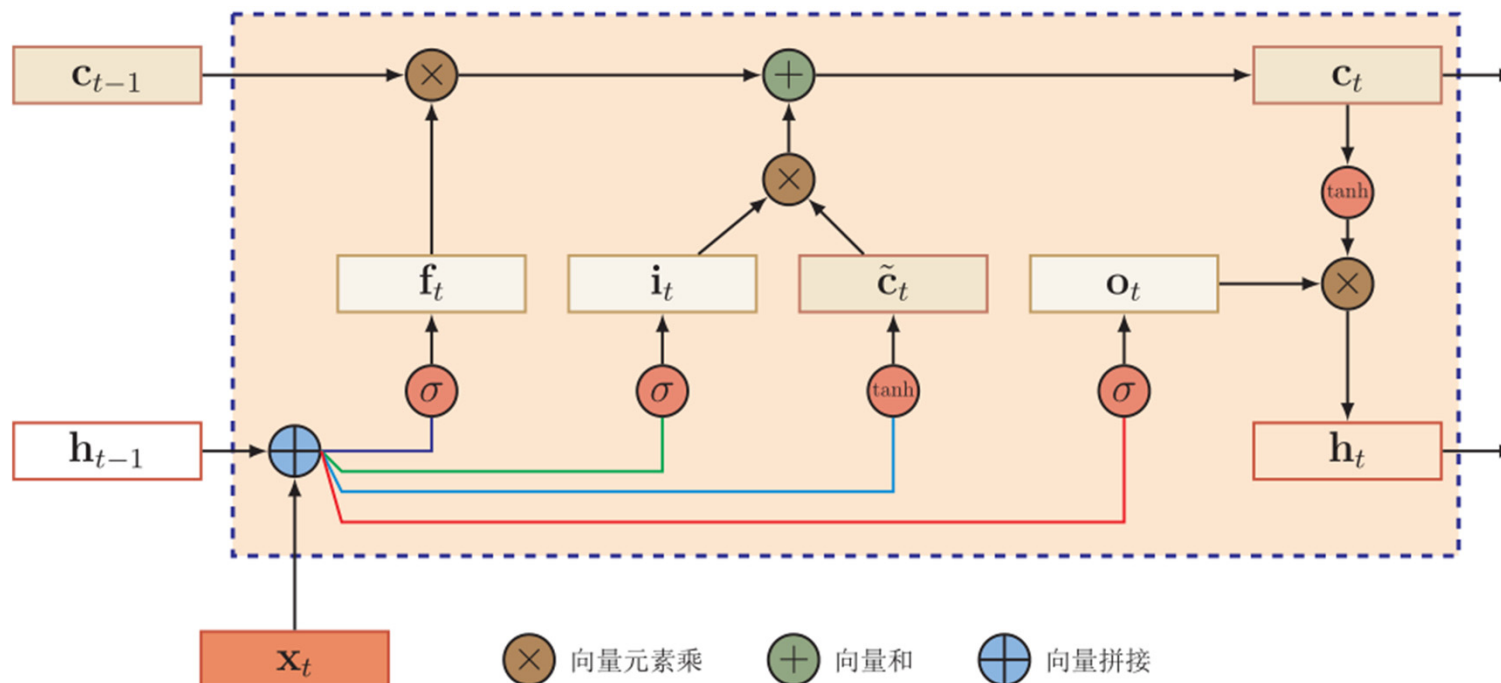
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o)$$

- ▶ 其中 $\mathbf{x}_t$ 为当前时刻的输入， $\mathbf{h}_{t-1}$ 为上一时刻的外部状态
- ▶ 下图给出了LSTM的循环单元结构，其计算过程：
 - ✓ 1) 利用上一时刻的外部状态 $\mathbf{h}_{t-1}$ 和当前时刻的输入 $\mathbf{x}_t$ ，计算三个门及候选状态 $\tilde{\mathbf{c}}_t$ ；
 - ✓ 2) 结合三门来更新记忆单元 $\mathbf{c}_t$ ；
 - ✓ 3) 结合输出门 $\mathbf{o}_t$ ，将内部状态的信息传递给外部状态 $\mathbf{h}_t$

1 长短期记忆神经网络：LSTM



$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i),$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f),$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o),$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$h_t = o_t \odot \tanh(c_t),$$

1 长短期记忆网络 LSTM

- ▶ 通过LSTM循环单元，整个网络可以建立较长距离的时序依赖关系。前述公式可以简洁的描述为

$$\begin{bmatrix} \tilde{\mathbf{c}}_t \\ \mathbf{o}_t \\ \mathbf{i}_t \\ \mathbf{f}_t \end{bmatrix} = \begin{bmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{bmatrix} \left(\mathbf{W} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{h}_{t-1} \end{bmatrix} + \mathbf{b} \right)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

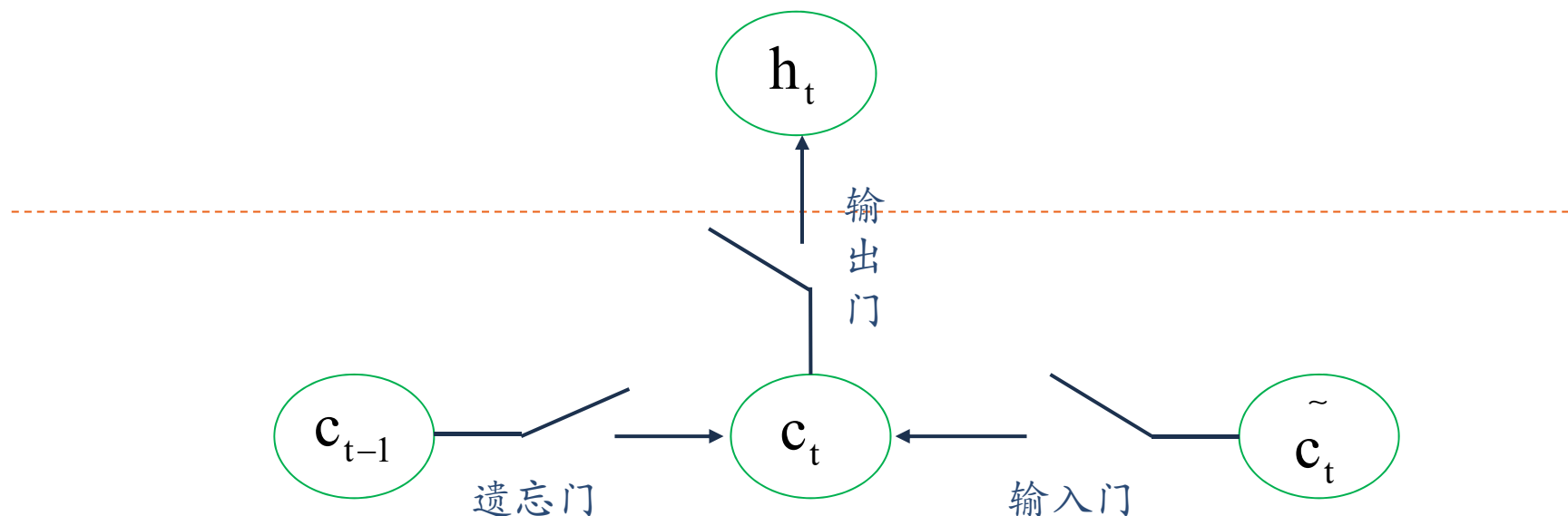
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- ▶ 其中 $\mathbf{W} \in \mathbb{R}^{4D \times (D+M)}$, $\mathbf{b} \in \mathbb{R}^{4D}$

1 长短期记忆网络 LSTM

▶ 3) 记忆

- ▶ 循环神经网络的隐状态 $\mathbf{h}_t$ 存储了历史信息，可以看作一种记忆
- ▶ 在简单循环网络中，隐状态每个时刻都会被重写，因此可以看作一种短期记忆 (Short-Term Memory)
- ▶ 在神经网络中，长期记忆 (Long-Term Memory) 可以看作网络参数，隐含了从训练数据中学到的经验，其更新周期要远远慢于短期记忆 $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$
- ▶ LSTM中，记忆单元 $\mathbf{c}$ 可以在某个时刻捕捉到某个关键信息，并有能力将此关键信息保存一定的时间间隔。记忆单元 $\mathbf{c}$ 中的保存信息的生命周期长于短期记忆 $\mathbf{h}$ (源自简单循环网络)，但又远远短于长期记忆 (源自神经网络)，因此称作长短期记忆



$$\begin{bmatrix} \tilde{\mathbf{c}}_t \\ \mathbf{o}_t \\ \mathbf{i}_t \\ \mathbf{f}_t \end{bmatrix} = \begin{bmatrix} \tanh \\ \sigma \\ \sigma \\ \sigma \end{bmatrix} \left(\mathbf{W} \begin{bmatrix} \mathbf{x}_t \\ \mathbf{h}_{t-1} \end{bmatrix} + \mathbf{b} \right)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

LSTM通过门控单元以及内部状态单元的线性自循环，给梯度的长距离持续流通提供了路径，改变了之前简单循环神经网络中信息和梯度的传播方式，解决了长程依赖问题

2 LSTM的各种变体

- ▶ 目前主流的LSTM网络使用三个门来动态地控制内部状态应该遗忘多少历史信息，输入多少新信息，输出多少信息。通过对门控机制的改进，可以获得LSTM的不同变体

- ▶ 没有遗忘门（最早的LSTM）

$$\boxed{\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t} \implies \mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- ▶ 耦合输入门和遗忘门

$$\mathbf{f}_t = 1 - \mathbf{i}_t$$

输入门和遗忘门互补，将二者合一

$$\mathbf{c}_t = (1 - \mathbf{i}_t) \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- ▶ peephole连接

三门不仅依赖于当前时刻的输入，还依赖于上一时刻的记忆单元

$$\begin{aligned} \mathbf{i}_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{V}_i \mathbf{c}_{t-1} + \mathbf{b}_i) \\ \mathbf{f}_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{V}_f \mathbf{c}_{t-1} + \mathbf{b}_f) \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{V}_o \mathbf{c}_{t-1} + \mathbf{b}_o) \end{aligned}$$

3 门控循环单元网络 GRU

► GRU是一种比LSTM简单的循环神经网络

- 与LSTM不同，GRU不引入额外的记忆单元，GRU引入了 **更新门** (Update Gate) 来控制当前状态需要从历史状态中保留多少信息，以及需要从候选状态中接受多少新信息

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta) \implies \mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$$

- $\mathbf{z}_t \in [0, 1]^D$ 为更新门 $\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z)$

- 在GRU中， $g(\mathbf{x}, \mathbf{h}_{t-1}; \theta)$ 定义为

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

- $\tilde{\mathbf{h}}_t$ 表示当前时刻的**候选状态**， $\mathbf{r}_t \in [0, 1]^D$ 为**重置门** (Reset Gate)

3 门控循环单元网络 GRU

- ▶ **重置门** r_t 的定义为

$$r_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r)$$

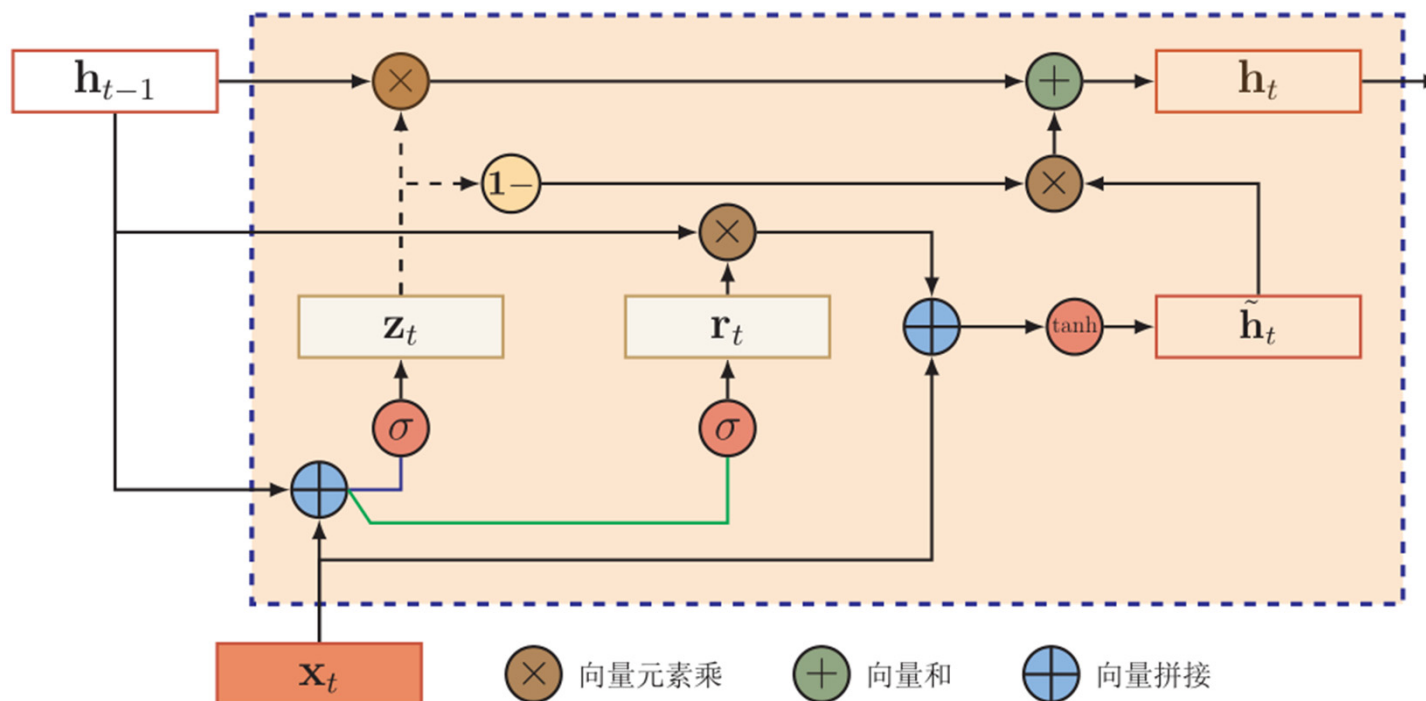
- ▶ 其用来控制候选状态 $\tilde{\mathbf{h}}_t$ 的计算是否依赖上一时刻的状态 $\mathbf{h}_{t-1}$
- ▶ 综上，GRU的状态更新方式为

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot g(\mathbf{x}, \mathbf{h}_{t-1}; \theta)$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$$

- ▶ 下图给出了GRU网络的循环单元结构

3 门控循环单元网络 GRU



重置门

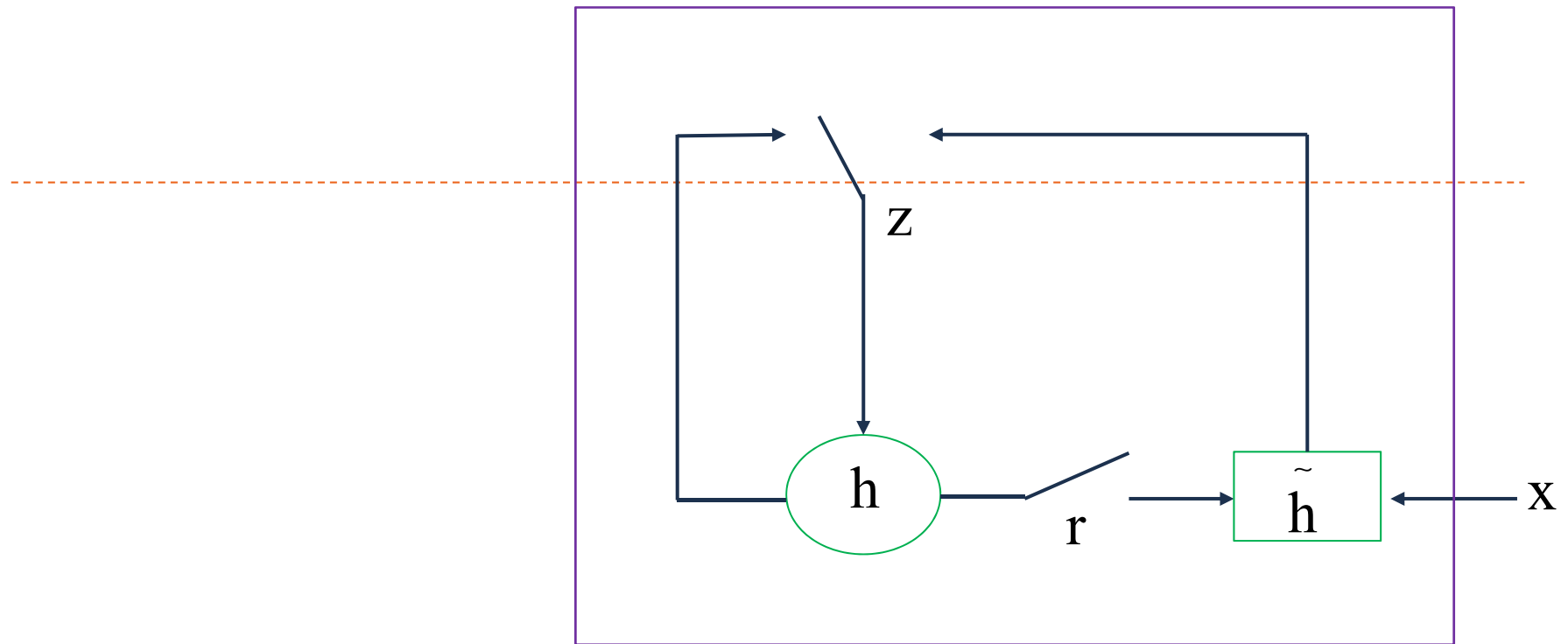
$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r),$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}(\mathbf{r}_t \odot \mathbf{h}_{t-1}))$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z),$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t,$$

更新门



$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z)$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$$

7 深层循环神经网络

1 堆叠循环神经网络

▶ 堆叠循环神经网络 (Stacked Recurrent Neural Network, SRNN), 即将多个循环神经网络堆叠起来

▶ 循环多层感知器, 即堆叠的简单循环神经网络

▶ 下图中, 第l层网络的输入是第l-1层网络的输出。我们定义 $\mathbf{h}_t^{(l)}$ 为在时刻t的第l层的隐状态

$$\mathbf{h}_t^{(l)} = f\left(\mathbf{U}^{(l)}\mathbf{h}_{t-1}^{(l)} + \mathbf{W}^{(l)}\mathbf{h}_t^{(l-1)} + \mathbf{b}^{(l)}\right)$$

▶ 其中, U、W、和b为权重矩阵和偏置向量 $\mathbf{h}_t^{(0)} = \mathbf{x}_t$

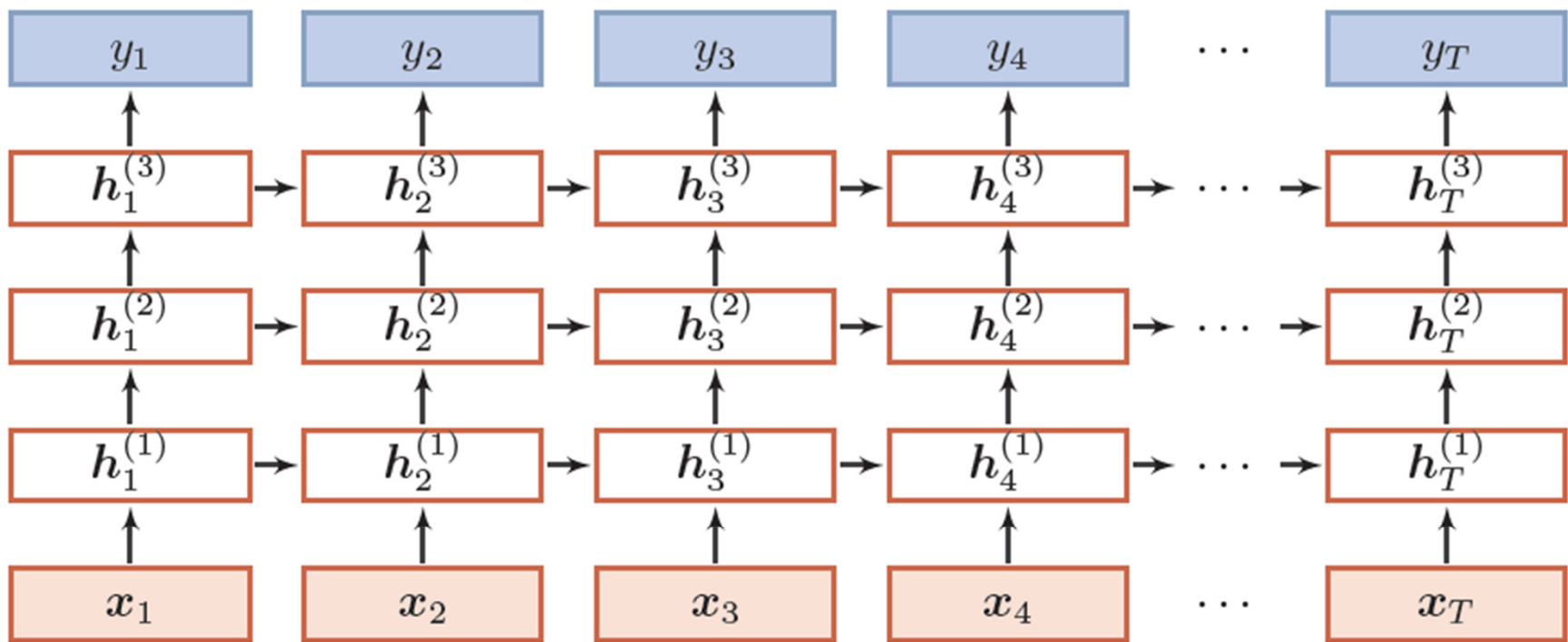


图 6.9 按时间展开的堆叠循环神经网络

2 双向循环神经网络

► 双向循环神经网络(Bi-RNN) ，由两层循环神经网络组成，输入相同，传递信息的方向不同

► 假设第1层按时间顺序，第2层按时间逆序，在时刻t时的隐状态定义为 $\mathbf{h}_t^{(1)}$ 和 $\mathbf{h}_t^{(2)}$ ，则

$$\mathbf{h}_t^{(1)} = f\left(\mathbf{U}^{(1)}\mathbf{h}_{t-1}^{(1)} + \mathbf{W}^{(1)}\mathbf{x}_t + \mathbf{b}^{(1)}\right)$$

$$\mathbf{h}_t^{(2)} = f\left(\mathbf{U}^{(2)}\mathbf{h}_{t+1}^{(2)} + \mathbf{W}^{(2)}\mathbf{x}_t + \mathbf{b}^{(2)}\right)$$

$$\mathbf{h}_t = \mathbf{h}_t^{(1)} \oplus \mathbf{h}_t^{(2)}$$

► 下图给出了按时间展开的双向循环神经网络

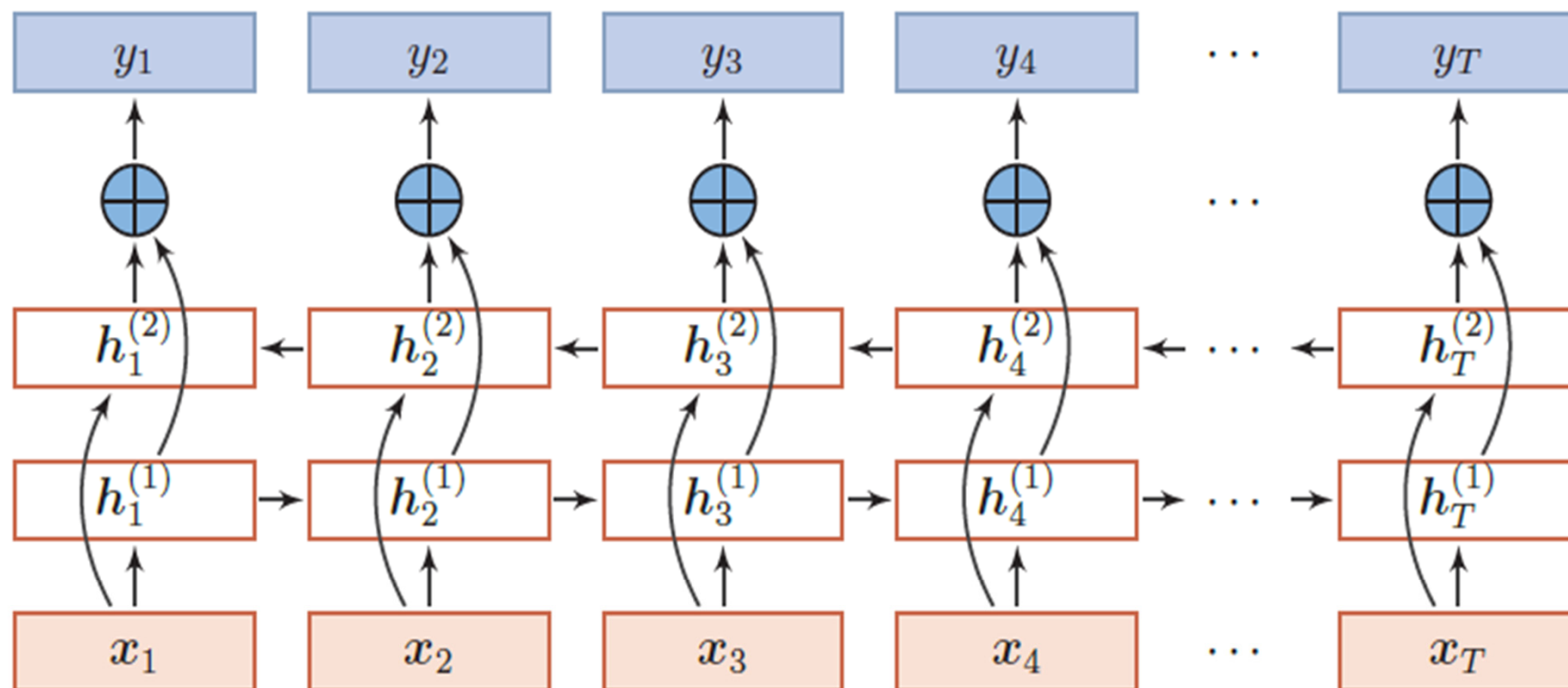


图 6.10 按时间展开的双向循环神经网络

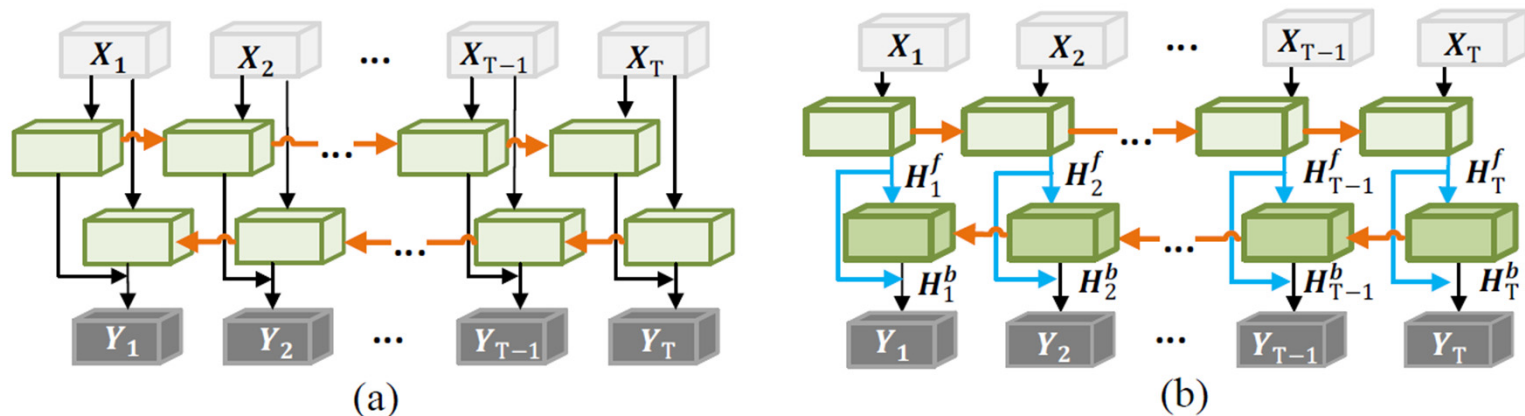


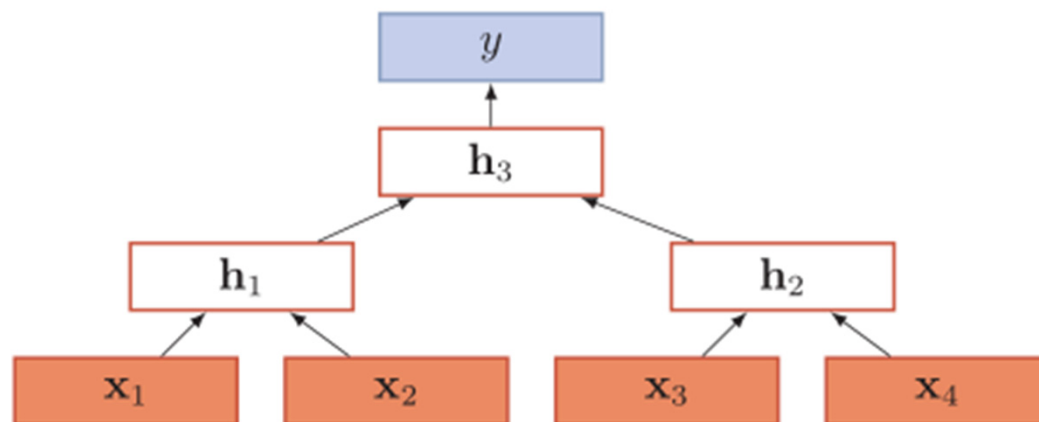
Fig. 3. Illustration of (a) Bidirectional ConvLSTM and (b) the proposed DB-ConvLSTM module. In PDB-ConvLSTM module, two DB-ConvLSTMs with different dilate rates are adopted for capturing multi-scale information and encouraging information flow between bi-directional LSTM units. See § 3.2 for details.

8 扩展到图结构

1 递归神经网络

Recursive Neural Network

- ▶ 递归神经网络是在一个有向无循环图上的扩展。
- ▶ 递归神经网络的一般结构为树状的层次结构，如下图所示
- ▶ 图中，三个隐藏层 h_1 h_2 h_3 ,其中 h_1 由两个输入层 x_1 和 x_2 计算得到， h_2 由 x_3 和 x_4 计算得到， h_3 由两个隐藏层 h_1 和 h_2 得到
- ▶ 对于一个节点 h_i ，它可以接受来自父节点 π_i 中所有节点的消息，并更新自己的状态 $\mathbf{h}_i = f(\mathbf{h}_{\pi_i})$
- ▶ 其中， $\mathbf{h}_{\pi_i}$ 表示集合 π_i 中所有节点状态的拼接， $f(\cdot)$ 是一个和节点位置无关的非线性函数。



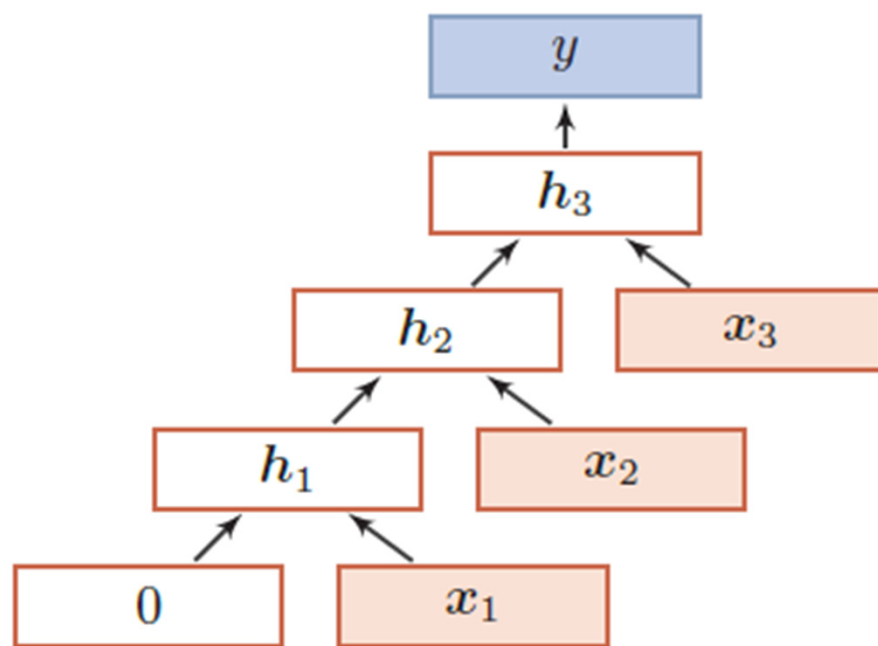
1 递归神经网络

Recursive Neural Network

- ▶ 上图所示的递归神经网络具体可以写为

$$\mathbf{h}_1 = \sigma\left(\mathbf{W}\begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} + \mathbf{b}\right), \mathbf{h}_2 = \sigma\left(\mathbf{W}\begin{bmatrix} \mathbf{x}_3 \\ \mathbf{x}_4 \end{bmatrix} + \mathbf{b}\right), \mathbf{h}_3 = \sigma\left(\mathbf{W}\begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \end{bmatrix} + \mathbf{b}\right)$$

- ▶ 其中 $\sigma(\cdot)$ 表示非线性激活函数， $\mathbf{W}$ 和 $\mathbf{b}$ 是可学习参数。
- ▶ 输出层可以作为一个分类器 $y = g(\mathbf{W}'\mathbf{h}_3 + \mathbf{b}')$
- ▶ 当递归神经网络的结构退化为线性序列时，如图所示，递归神经网络等价于简单循环网络



(b) 退化结构

2 图网络

- ▶ 在实际应用中，很多数据是图结构的，比如知识图谱、社交网络、分子网络等。而前馈网络和循环网络很难处理图结构的数据。
- ▶ 图神经网络GNN是将消息传递的思想扩展到图结构数据上的神经网络

2 图网络

► 对于一个任意的图结构 $G(V, E)$

- 图中每个节点 v 都用一组神经元来表示其状态 $\mathbf{h}^{(v)}$ ，初始状态可以是节点 v 的输入特征 $\mathbf{x}^{(v)}$ 。

- **更新函数**，即每个节点可以收到来自相邻节点的消息，

并更新自己的状态
$$\mathbf{m}_t^{(v)} = \sum_{\mu \in N(v)} f\left(\mathbf{h}_{t-1}^{(v)}, \mathbf{h}_{t-1}^{(\mu)}, \mathbf{e}^{(u,v)}\right)$$

$$\mathbf{h}_t^{(v)} = g\left(\mathbf{h}_{t-1}^{(v)}, \mathbf{m}_t^{(v)}\right)$$

- 其中 $N(v)$ 表示节点 v 的邻居， $\mathbf{m}_t^{(v)}$ 表示在第 t 时刻节点 v 收到的信息， $\mathbf{e}^{(u,v)}$ 为边 e 上的特征

2 图网络

- **读出函数**，即在整个图更新T次之后，可以通过一个读出函数 $g(\cdot)$ 来得到整个网络的表示：

$$\mathbf{o}_t = g\left(\left\{\mathbf{h}_T^{(v)} \mid v \in V\right\}\right)$$

9循环网络应用

语言模型

► 自然语言理解 → 一个句子的可能性/合理性

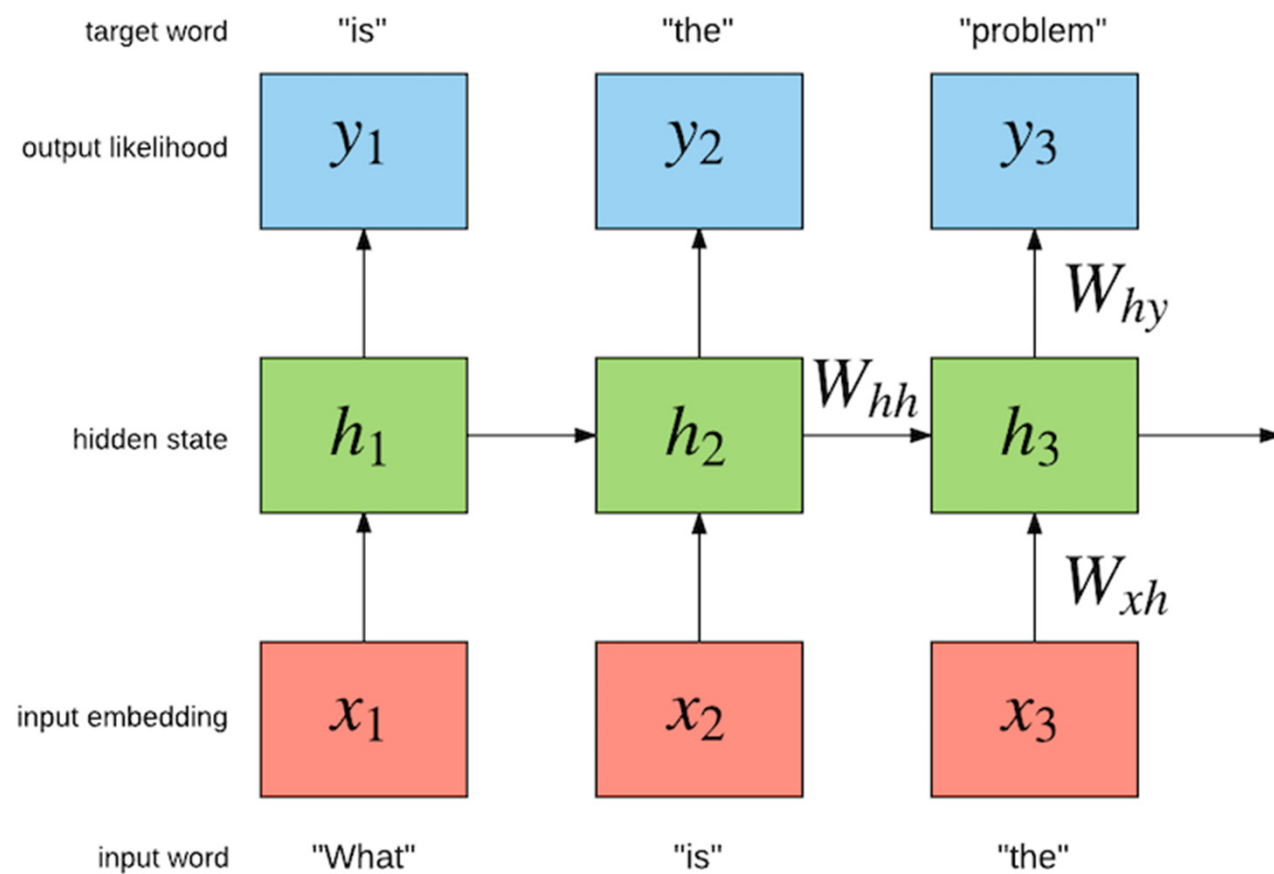
- ! 在报那猫告做只
- 那只猫在作报告!
- 那个人在作报告!



► 一切都是概率!

- $P(x_1, x_2, \dots, x_n)$
- $= \prod_i P(x_i | x_{i-1}, \dots, x_1)$
- $\approx \prod_i P(x_i | x_{i-1}, \dots, x_{i-n+1})$ N元语言模型

语言模型



作词机

► RNN在“学习”过汪峰全部作品后自动生成的歌词

► <https://github.com/phunterlau/wangfeng-rnn>

我在这里中的夜里
就像一场是一种生命的意叶
就像我的生活变得在我一样
可我们这是一个知道
我只是一天你会怎吗
可我们这是我们是不要为你
我们想这有一种生活的时候

作诗

白鹭窥鱼立， Egrets stood, peeping fishes. 青山照水开。 Water was still, reflecting mountains. 夜来风不动， The wind went down by nightfall, 明月见楼台。 as the moon came up by the tower.	满怀风月一枝春， Budding branches are full of romance. 未见梅花亦可人。 Plum blossoms are invisible but adorable. 不为东风无此客， With the east wind comes Spring. 世间何处是前身。 Where on earth do I come from?
--	--

传统统计机器翻译

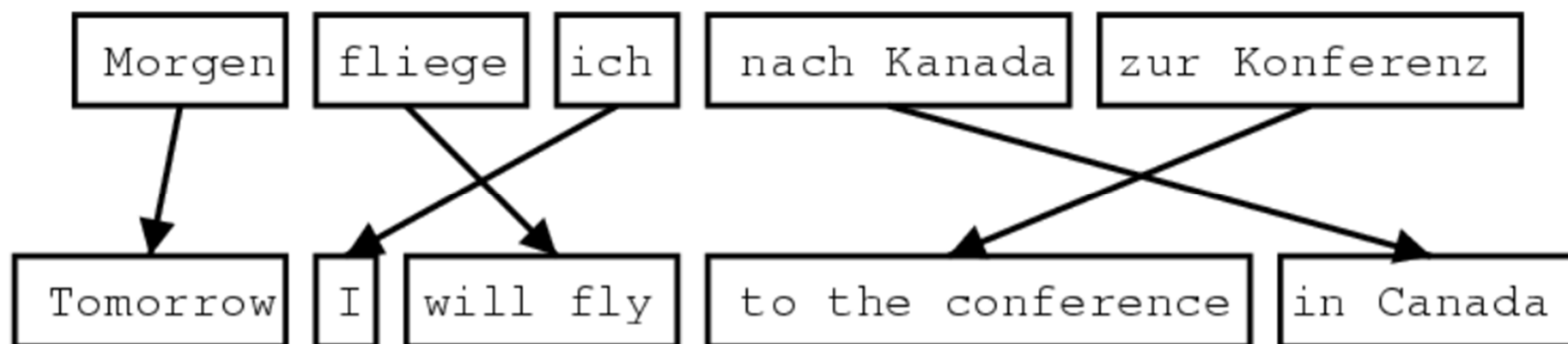
► 源语言: f

► 目标语言: e

► 模型: $\hat{e} = \operatorname{argmax}_e p(e|f) = \operatorname{argmax}_e p(f|e)p(e)$

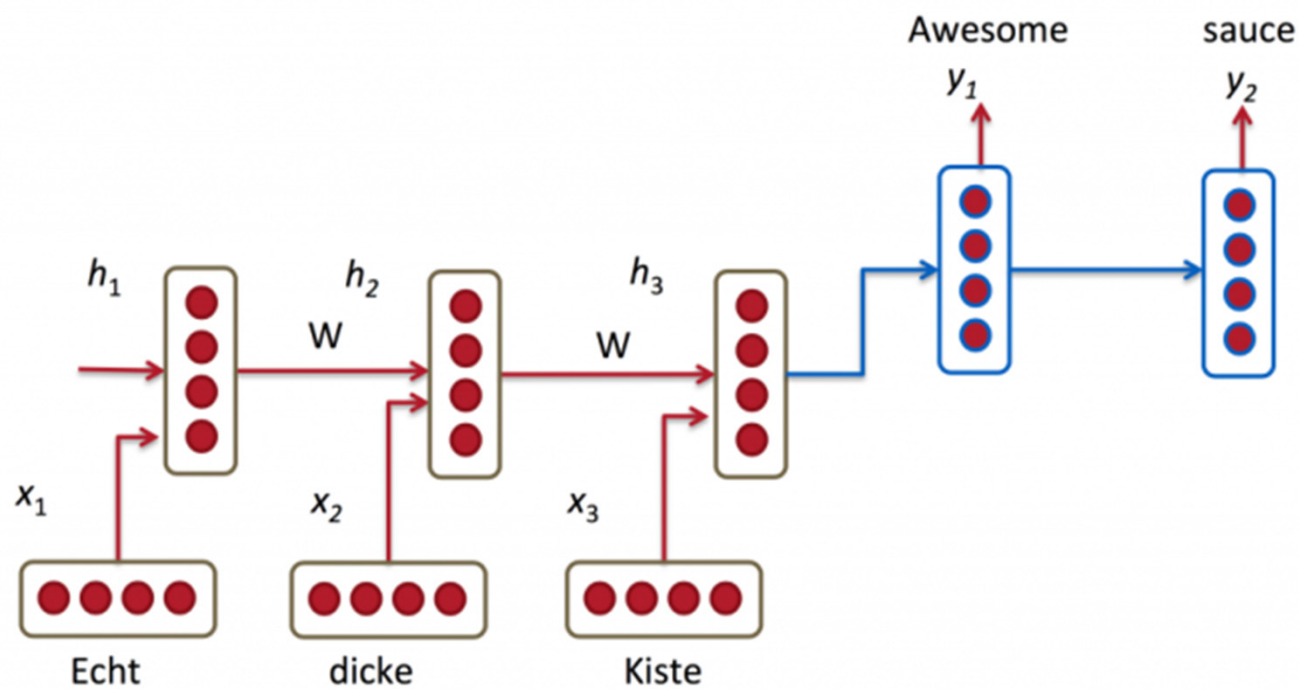
► $p(f|e)$: 翻译模型

► $p(e)$: 语言模型

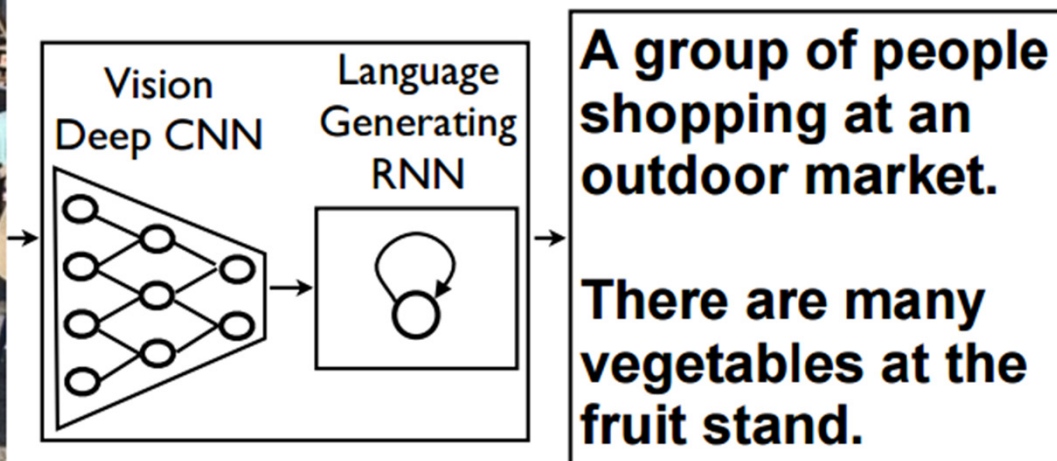


基于序列到序列的机器翻译

- ▶ 一个RNN用来编码
- ▶ 另一个RNN用来解码



看图说话 Image Caption



看图说话

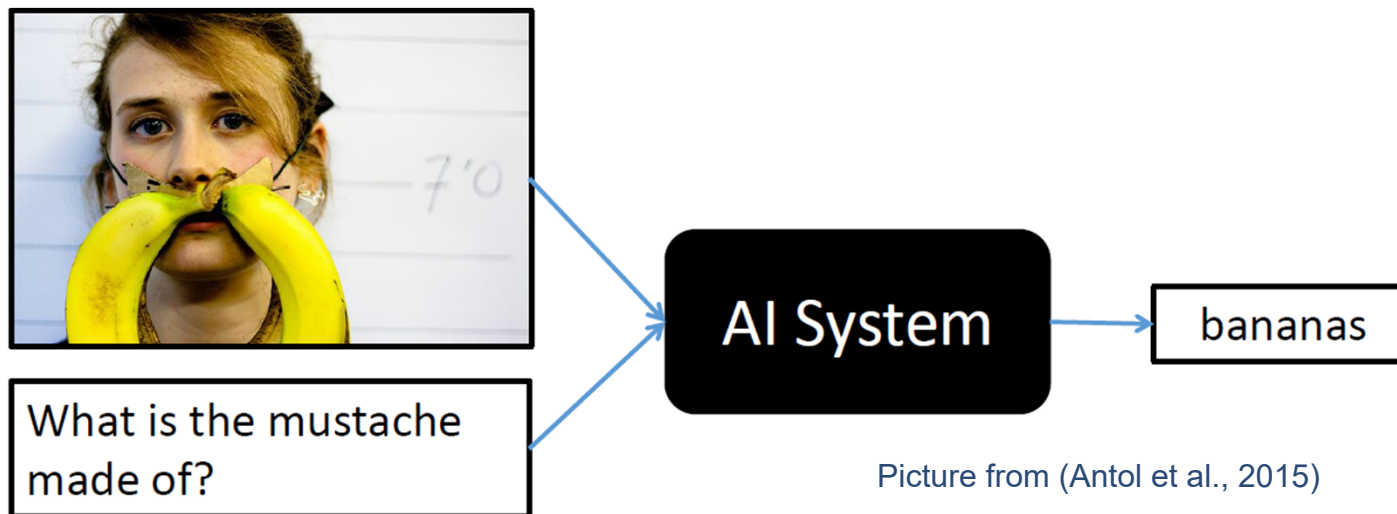


Figure 5. A selection of evaluation results, grouped by human rating.

Visual Question Answering (VQA)

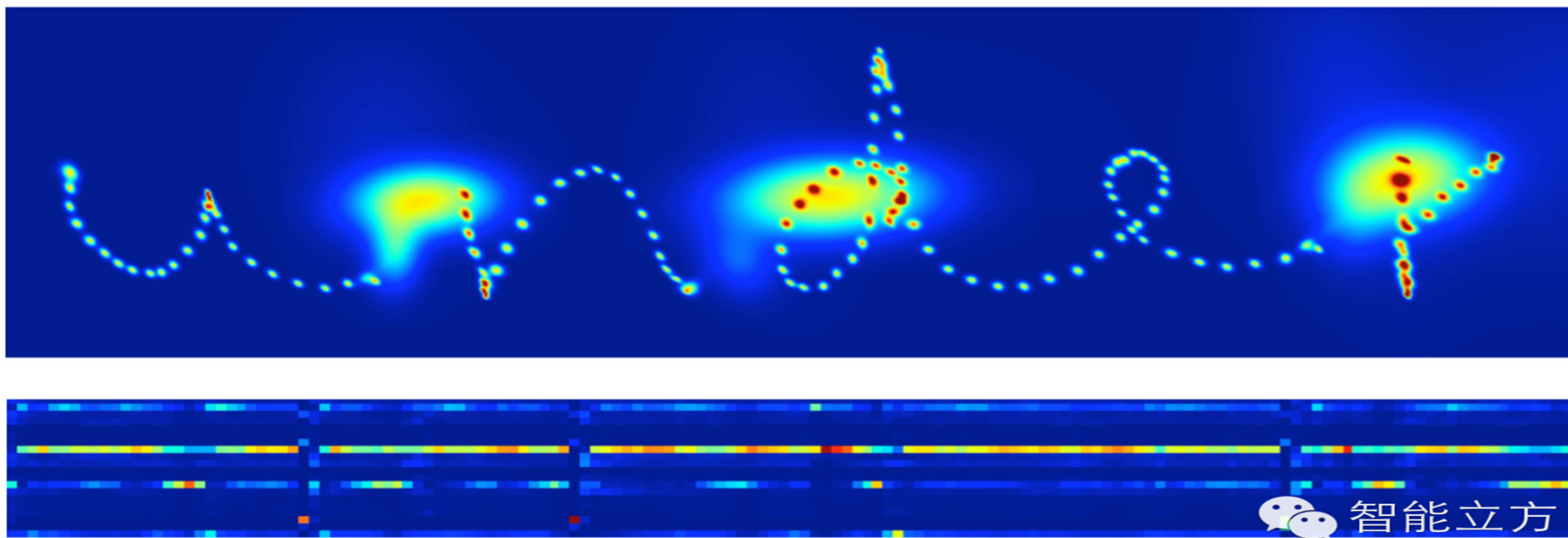
Demo Website

VQA: Given an image and a natural language question about the image, the task is to provide an accurate natural language answer



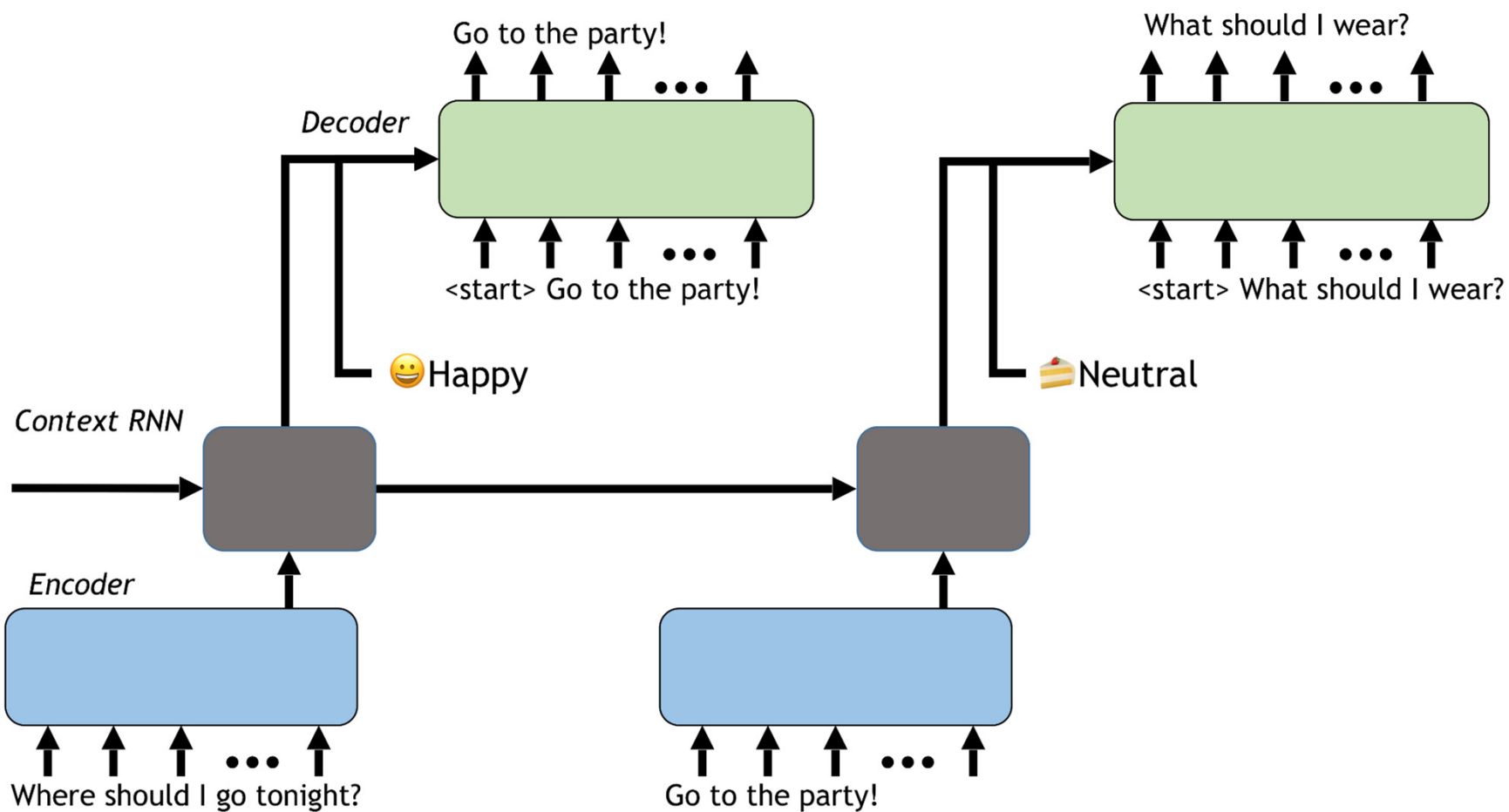
写字

- ▶ 把一个字母的书写轨迹看作是一连串的点。一个字母的“写法”其实是每一个点相对于前一个点的偏移量，记为 $(\text{offset } x, \text{offset } y)$ 。再增加一维取值为0或1来记录是否应该“提笔”。



对话系统

<https://github.com/lukalabs/cakechat>



10 循环神经网络总结

▶ 优点:

▶ 引入记忆

- ▶ 延时神经网络/有外部输入的非线性自回归模型/循环神经网络

▶ 图灵完备

- ▶ 简单循环网络、通用近似定理（完全连接的循环神经网络）

▶ 应用到机器学习

- ▶ 序列到类别/同步的序列到序列/异步的序列到序列

▶ 缺点:

▶ 长程依赖问题

- ▶ 梯度消失、梯度爆炸

▶ 记忆容量问题

▶ 并行能力

课后小练习

▶ 编程练习

- ▶ <https://github.com/nndl/exercise/>
- ▶ [chap6_RNN](#)
- ▶ 利用循环神经网络来生成唐诗

谢 谢